Vulnerability Scan Report

Field	Value
Project	deepsec-default
Date	2026-08-31T12:40:38.646Z
Files tracked	278
Files analyzed	278
Total findings	71

Summary

Severity	Count
CRITICAL	0
HIGH	2
MEDIUM	25
HIGH_BUG	4
BUG	40

HIGH (2)

loadEnvFile merges a file living inside the untrusted scanned repo (/.env.local) into process.env with no key allowlist or trust check, enabling repo-controlled credential redirection

- **File:** `packages/deepsec/src/env-file.ts`
- **Lines:** 58, 60, 68
- **Slug:** other-env-injection
- **Confidence:** medium

loadEnvFile(filePath, target = process.env) (L58-70) does `Object.assign(target, parsed)` (L68) with no validation of key names (unlike updateEnvFile, which at least validates ENV_NAME) and no check that the file's location is operator-trusted. Its call sites load `path.join(workspaceDir, '.env.local')` — and the default deepsec workspace is `.deepsec/` INSIDE the scanned repository (init.ts: workspaceArg defaults to '.deepsec'; docs/faq.md confirms the workspace lives in the scanned repo and is 'checked into git'). Critically, `deepsec init` silently resumes a pre-existing workspace when it contains `deepsec.config.ts` plus `data/<basename>/project.json` (init.ts resume path), all of which an attacker can commit, and nothing in the skeleton write ever overwrites a pre-existing `.env.local`. The callers wire the parsed file straight into the live process env: setup/coordinator.ts:326 passes `env: process.env` to ensureConnectedWorkspace, which calls ensureVercelLink → `loadEnvFile(join(workspaceDir, '.env.local'), env)` (vercel-link.ts:306), and setup/plan.ts:80 loads it into the env copy used for

credential decisions. Attack scenario: a malicious repository ships `.deepsec/{deepsec.config.ts, data/<repo-name>/project.json (rootPath '.'), .env.local}`; a victim following the documented quickstart (`cd <repo> && npx deepsec init`) silently resumes the attacker's workspace and the attacker-chosen values are injected into the scanner's environment — including `AI_GATEWAY_API_KEY` (all model traffic — i.e. the scanned source and every finding — flows through the attacker's gateway account, exfiltrating scan data and routing spend), `ANTHROPIC_BASE_URL`/`OPENAI_BASE_URL` (resolveModelRoute 'direct' mode sends the victim's real provider API key to any https URL, so a repo-set base URL exfiltrates the credential), `VERCEL_TOKEN`/`VERCEL_TEAM_ID`/`VERCEL_PROJECT_ID` (the workspace is linked to the attacker's Vercel project), `DEEPSEC_DATA_ROOT`, `DEEPSEC_DATA_BRANCH`, and `DEEPSEC_INSIDE_SANDBOX`. Because Object.assign overwrites, loadEnvFile's values even beat pre-existing operator env in the vercel-link path. The same class of injection is also reachable through cli.ts's startup dotenv of the repo-root `.env`/`.env.local`; env-file.ts is the shared loader used by the setup/auth paths and lacks any mitigation (allowlist of known deepsec keys, trust check on the workspace path, or refusing keys that steer credential routing).

Recommendation: Restrict loadEnvFile to an allowlist of deepsec-managed keys (or at minimum deny credential/route-steering names such as AI_GATEWAY_API_KEY, *BASE_URL, ANTHROPIC*, OPENAI*_, VERCEL_TOKEN, DEEPSEC_DATA*_), validate keys with ENV_NAME like updateEnvFile does, refuse to load `.env.local` from a workspace that was not created/owned by this installation (e.g. verify provenance recorded at init time), and never let repo-shipped env values override operator-provided process.env values (merge with precedence for existing vars).

Vercel CLI credential probe executes with cwd set to the untrusted scanned repository

- **File:** `packages/deepsec/src/setup/plan.ts`
- **Lines:** 89, 92, 93
- **Slug:** rce
- **Confidence:** medium

buildSetupPlan() runs the auth probe as runPinnedVercelCli(["whoami"], projectRoot) (L89-93), where projectRoot is the repository being scanned (init.ts passes targetAbs from the user-supplied , and the plan command is explicitly an inspection step agents run against arbitrary checkouts). Every other invocation site in the codebase deliberately uses the deepsec-owned workspaceDir as cwd; this one alone points the spawned process at the untrusted repo. runPinnedVercelCli shells out to `npx --yes vercel@56.3.2 whoami`, and npm/npx read .npmrc from the invocation cwd and run dependency lifecycle scripts on a cache miss, so a malicious repository can ship a project-level .npmrc (registry/auth-token substitution) or node_modules/prepare hooks that execute attacker code when the plan probe runs. That code inherits the host's full environment (VERCEL_TOKEN, AI_GATEWAY_API_KEY, or an authenticated vercel CLI state), turning a routine `deepsec init --plan` against a cloned repo into credential theft or RCE on the analyst's machine. This violates the threat model's stated convention that argv arrays, allowlists, and workspace-scoped cwds are the control set for every spawn site.

Recommendation: Probe CLI authentication with cwd set to workspaceDir (or a neutral temp directory) exactly like every other runPinnedVercelCli call site in vercel-link.ts, never with projectRoot; additionally consider passing a sanitized env and --ignore-scripts to npx for unattended probes.

MEDIUM (25)

Privileged comment job posts PR-influenced artifact content verbatim as the github-actions bot

- **File:** `.github/workflows/deepsec.yml`
- **Lines:** 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75
- **Slug:** other-untrusted-artifact-post
- **Confidence:** low

The analyze job runs PR-controlled code (the bundled deepsec CLI plus AI agents reading PR content) and writes comment.md; the comment job — the only job with pull-requests: write — then posts that file's raw contents via github-script's issues.createComment with no sanitization or length/schema validation. A PR author therefore controls text that is displayed under the trusted github-actions[bot] identity on the PR (e.g., convincing 'fix required, re-auth at ...' text, tracking-image markdown, or @mentions that fire notifications to maintainers/org members). Impact is bounded: issue_number/owner/repo come from workflow context, so the comment can only target the attacker's own PR, and same-repo PR authors already have repo write access (so no permission escalation). Still, the workflow's stated design goal — 'the privileged step never has to run any PR-controlled code' — is undermined by trusting PR-controlled data in the privileged step.

Recommendation: Treat comment.md as untrusted: validate it against a strict schema (e.g., JSON produced by deepsec with only finding titles/paths/diff stats), render the bot comment from trusted templates rather than raw file content, strip/escape markdown constructs (raw HTML, images, @mentions), and cap length before posting.

---

bash -x shell tracing writes the secret MODEL_BASE_URL into TaskRun/pod logs

- **File:** `noc/pipeline.yaml`
- **Lines:** 51, 53, 78
- **Slug:** secret-in-log
- **Confidence:** medium

The pre-prepare step runs 'bash -x /home/scan/deepsec-setup.sh' (L78). With xtrace enabled, bash prints every command after parameter expansion to stderr, which Tekton captures in TaskRun logs (readable by anyone with namespace pod-log access and typically retained by log aggregation). The setup script's final command expands the value injected from Secret 'llm-access' key MODEL_BASE_URL (L51-55) directly into the traced command line: 'npx deepsec init ... --ai-base-url $MODEL_BASE_URL ...' — so the secret's literal value is written verbatim into the logs. Because MODEL_BASE_URL is stored as a Kubernetes Secret alongside LLMAPIKEY (rather than a ConfigMap), it is by definition credential-bearing or otherwise sensitive. The same trace also expands $MODEL_NAME, $REPO_DIR, $WORK_DIR and $PROJECT_ID. The Containerfile's placeholder (ENV MODEL_BASE_URL='https://maas.example.com/v1') confirms this variable is expected to carry sensitive endpoint configuration.

Recommendation: Remove the '-x' flag (or add targeted, redacted logging). Pass the base URL to deepsec via a config file or env var consumed internally rather than an expanded CLI argument, so no command line ever contains the secret value.

LLM credentials injected into the environment of the step that processes untrusted repo content, with no egress restriction

- **File:** `noc/pipeline.yaml`
- **Lines:** 48, 51, 56, 83, 89
- **Slug:** other-prompt-injection-egress
- **Confidence:** medium

The scan task's stepTemplate (L48-63) injects LLMAPIKEY and MODEL_BASE_URL from Secret 'llm-access' into the environment of every step, including the repo-scan step (L83-96), where the deepsec coding agent reads and executes tooling against a repository cloned from the caller-supplied 'repo-url' pipeline param. deepsec's own threat model treats scanned repos as untrusted input (prompt injection via source comments reaches agents with shell/read access) and mitigates this in its primary deployment by running agents in sandboxed VMs with egress restricted through a loopback proxy that only reaches model gateways. This manifest applies no NetworkPolicy or egress control, so a repo that successfully prompt-injects the agent (or any child process inheriting the step env) can exfiltrate the LLM API credentials and the scanned repository's contents to arbitrary attacker-controlled endpoints directly from the pod. The pipeline-runs sample shows the pipeline is intended to scan arbitrary repos (e.g. github.com/noelo/deepsec).

Recommendation: Apply a NetworkPolicy/egress proxy restricting the scan pod to the model gateway (mirroring deepsec's sandbox design), avoid placing credentials in the stepTemplate env of the step that executes untrusted content, and run the agent via deepsec's sandbox mode rather than directly in the cluster pod.

Unpinned ':latest' runner image in a pipeline that hands it LLM secrets and untrusted repo content

- **File:** `noc/pipeline.yaml`
- **Lines:** 72, 89, 120
- **Slug:** other-supply-chain
- **Confidence:** low

All three steps run 'quay.io/noeloc/deepsec-runner:latest' (L72, L89, L120) with no digest pin. This image is executed with LLMAPIKEY/MODEL_BASE_URL in its environment and with shell access to the cloned repository. A mutated or maliciously retagged ':latest' (compromise of the quay.io/noeloc namespace, or tag squatting after a registry namespace lapse) silently replaces the code that runs with those credentials — arbitrary code execution in the cluster with secret access. Nothing in the manifest pins or verifies the image.

Recommendation: Pin the image by immutable digest (image: quay.io/noeloc/deepsec-runner@sha256:...) and enforce verification (e.g. admission policy/signature check) in the cluster.

Untrusted repo-controlled branch name and remote URL persisted unvalidated and interpolated unescaped into export markdown

- **File:** `packages/core/src/run.ts`
- **Lines:** 39, 45, 54, 72, 86
- **Slug:** other-markdown-injection

- **Confidence:** low

detectGithubUrl() runs `git rev-parse --abbrev-ref HEAD` and `git remote get-url origin` inside the scanned repository (L39-50) — a repo the threat model explicitly treats as untrusted input — and stores the result as `${https}/blob/${branch}` in data//project.json (L72, L86) with no validation beyond a substring check for 'github.com' (which a URL like https://github.com.evil.com/... also satisfies). Git ref-name rules forbid spaces and ~^:?*[\ but explicitly allow '(', ')', '"', '<', '>', ';', '=', and '|', so a malicious repo's default branch name (e.g. `main)(evil)` or `x">  `) becomes attacker-controlled data in project.json. projectConfigSchema only checks it is a string. Later, packages/deepsec/src/commands/export.ts interpolates this value unescaped: makeGithubLink() buils `${base}/blob/${branch}/${filePath}${anchor}` and buildDescription() emits `**File:** [`$record.filePath$`]({githubUrl})`. Since neither the URL nor the repo-derived filePath is markdown-escaped or URL-encoded, a branch name or filename containing ')' (or ']' for the backtick code span) terminates the markdown link early, letting the attacker inject arbitrary markdown into the generated report description (exported via md-dir markdown files and JSON issue payloads with labels/assignees). Scenario: a security team scans an untrusted third-party repo and exports findings to an issue tracker; the repo author manipulates report content — spoofing severities/text, injecting misleading or attacker-chosen links, or injecting HTML/images in renderers that don't sanitize. Impact is bounded (content injection in reports about the attacker's own repo, renderer-dependent execution), hence MEDIUM rather than HIGH.

Recommendation: Validate/normalize the detected githubUrl before persisting it (e.g. parse with URL, require origin === 'https://github.com', and reject/re-encode path segments), and when building export markdown, encodeURIComponent() the branch and filePath segments of the link and escape markdown-significant characters in link text/URLs.

PR-comment artifact embeds unsanitized AI/repo-derived text destined for GitHub rendering

- **File:** `packages/deepsec/src/commands/process.ts`
- **Lines:** 371, 372, 373, 374, 375, 376, 377, 378, 379, 380
- **Slug:** other-markdown-injection
- **Confidence:** medium

In direct mode, --comment-out renders a PR comment via renderPrComment() (packages/deepsec/src/pr-comment.ts) and writes it to disk for a GitHub workflow to post (lines 371-382). The rendered markdown interpolates finding free-text fields (title, description, recommendation) without any escaping. Per the project's stated threat model, the scanned repo is untrusted input and prompt injection from source comments reaches the AI agents that author these findings; a malicious repo can therefore steer finding title/description content. The resulting markdown is auto-posted to GitHub PRs (github-script), where images and links are rendered (script is sanitized): a crafted finding can inject phishing links ('click here to apply the fix' -> attacker site), fake verdicts/recommendation text, or tracking beacons, deceiving reviewers who act on the security report. Note the actual string interpolation sink lives in pr-comment.ts; this command is the flow that generates and exports the artifact. The scanner's [insecure-crypto] flag at line 238 is a false positive (the regex matched 'des' inside the word 'includes' in a docstring).

Recommendation: Escape or strip markdown/HTML control characters (backticks, brackets, image/link syntax, raw HTML) from AI-derived free-text fields before interpolating them into PR-comment/report

markdown, or render them inside fenced code blocks. Apply the same treatment in pr-comment.ts and report.ts.

Generated report.md interpolates attacker-influenced free text without escaping (content injection)

- **File:** `packages/deepsec/src/commands/report.ts`
- **Lines:** 57, 61, 62, 63, 64, 66, 73, 79, 80
- **Slug:** other-markdown-injection
- **Confidence:** medium

generateMarkdown() builds report.md by directly interpolating: f.title (L57), recent committer names/emails from gitInfo (L61-64), f.vulnSlug (L66), revalidation reasoning (L73), f.description (L79) and f.recommendation (L80). None of these values are escaped or sanitized. Under the project's threat model the scanned repo is untrusted, and two of these channels are attacker-controlled without requiring any prompt-injection success: (1) recentCommitters come straight from `git log` of the scanned repository, and git author/committer name/email are arbitrary attacker-chosen strings in a malicious repo; (2) finding title/description/recommendation/reasoning are free-form AI output (zod validates only that they are strings) steerable via prompt injection planted in repo content; (3) in sandbox runs, sandbox-produced records merge into host data after schema validation but with free-text fields intact. report.md is a security artifact typically pasted into GitHub PRs/issues or rendered in markdown viewers; GitHub renders images and links (script is stripped), so injected content can produce phishing links, spoofed verdicts ('~~false-positive~~' styling, fake 'confirmed fixed' text, misleading recommendations), or hidden content that deceives the human triaging the report. The scanner's [insecure-crypto] flag at line 79 is a false positive (the regex matched 'des' inside 'f.description').

Recommendation: Escape markdown-significant characters (or HTML-encode for HTML sinks) in all interpolated free-text fields — title, description, recommendation, reasoning, vulnSlug, and git committer name/email — or confine them to fenced code blocks. Also consider validating/normalizing git author strings at enrich time.

Credential scrub only covers candidates[].snippet — agent-authored findings text, reports/, and runs/ (modelConfig incl. aiHeaders) are committed to the data repo unredacted

- **File:** `packages/deepsec/src/data-commit.ts`
- **Lines:** 73, 77, 78, 88, 93, 133
- **Slug:** other-secrets-in-findings
- **Confidence:** medium

scrubCommittedDataDir() redacts `candidates[].snippet` for SECRET_SLUGS (L88-95) and fail-closes on leftover candidate snippets matching CREDENTIAL_RE, but it explicitly processes only JSON files under a `files/` path segment (L77-78: `if (!p.includes(`$path.sep files${path.sep}`)) continue;`) and only inspects `rec.candidates`. Everything else that `git add -A` (L133) stages into the data repo is committed verbatim: (1) FileRecords' `findings[]` — free-text `description`, `title`, `recommendation`, `triage.reasoning`, and `revalidation.reasoning` generated by the AI agent after reading the scanned source. For `secrets-exposure` / `secret-in-fallback` / `secret-in-log` candidates, agent findings routinely quote the credential verbatim ('the hardcoded key sk_live_... is...'), which is the most likely

carrier of a real secret into the committed data — yet the processor prompt contains no instruction to redact secrets from findings, and the write path has no redaction (only setup-reporter events are redacted via createSetupRedactor). (2) `runs/<runId>.json` (RunMeta) and `analysisHistory[].modelConfig` — the raw buildAgentConfig output is persisted as `processorConfig.modelConfig` / `modelConfig` (processor/src/index.ts L308/L714/L1342), which includes `aiHeaders` verbatim; an operator-supplied `--ai-header 'Authorization=Bearer <token>'` (or any credential-bearing custom header) is therefore written into committed JSON unredacted. (3) `reports/` renderings of the same findings. Result: real credentials lifted from the scanned repo can be pushed to the data repo's origin (the README itself warns against pointing it at a public repo). The README documents two accepted denylist gaps for *candidate snippets* (new slugs, regex misses), but this is a distinct structural gap: the scrub's file/field scope excludes the artifacts most likely to contain verbatim secrets.

Recommendation: Extend scrubCommittedDataDir to all JSON under DATA_DIR (runs/, reports/, project.json) and run the CREDENTIAL_RE sweep over every string field of findings (description, title, recommendation, triage.reasoning, revalidation.reasoning) and analysisHistory[].modelConfig, not just candidates[].snippet. Structurally, redact at write time: strip credential-shaped substrings from all persisted free text in writeFileRecord, and never persist credential-bearing aiHeaders into modelConfig.

DEEPSEC_DATA_BRANCH regex allows a leading '-' — value is passed to `git pull` where it can be parsed as a git option (e.g. --upload-pack)

- **File:** `packages/deepsec/src/data-commit.ts`
- **Lines:** 11, 12, 156
- **Slug:** other-git-option-injection
- **Confidence:** low

REF_RE = /^[A-Za-z0-9._/-]+$/ (L11) permits values beginning with '-', and RAW_BRANCH (L12) is passed as a bare positional to `execFileSync('git', ['pull', '--rebase', 'origin', DATA_BRANCH])` (L156). git-pull's parse-options permutes arguments, so a value like `--upload-pack=<cmd>` placed after `origin` is still parsed as a fetch option; with a local-path origin the '' is executed locally by the transport (with https/ssh origins it is passed to the remote side). The comment above the regex claims the validation makes the value safe, but it only excludes shell metacharacters — which are already neutralized by the argv-array invocation — while permitting git option injection. DEEPSEC_DATA_BRANCH is operator env in the normal threat model, so this is not directly exploitable on its own; it becomes reachable when an attacker controls the process environment, e.g. via a repo-shipped `.deepsec/.env.local` merged by loadEnvFile (see the env-file.ts finding) or a poisoned workspace, at which point `deepsec enrich` would execute an attacker-chosen command if the data repo's origin is a local path. Defense-in-depth issue rather than a standalone vulnerability.

Recommendation: Anchor the ref regex to forbid a leading dash and leading/trailing slashes, e.g. /^[A-Za-z0-9][A-Za-z0-9._/-]*$/ plus a `--`-style disambiguation where git supports it, or validate against `git check-ref-format --branch`. Also ensure DEEPSEC_DATA_BRANCH cannot originate from repo-controlled .env files.

Unescaped attacker-influenceable finding text and file paths interpolated into PR-comment markdown

- **File:** `packages/deepsec/src/pr-comment.ts`
- **Lines:** 100, 108, 110, 112, 116, 120
- **Slug:** other-markdown-injection
- **Confidence:** medium

renderPrComment() builds the PR-comment markdown by interpolating finding fields and file paths directly into the output with no escaping or markdown sanitization: `source` (line 100), file.filePath (line 108), finding.title (line 110), finding.vulnSlug (line 112), finding.description (line 116) and finding.recommendation (line 120). The trust chain is hostile: deepsec's threat model states the scanned repo is untrusted and that prompt injection via source content reaches the AI agents. The model-emitted Finding JSON is only shape-validated (core's findingSchema allows arbitrary strings for title/description/recommendation/vulnSlug — no content restrictions), so content embedded in a scanned file or PR diff (e.g. a comment instructing the model to emit a specific description) flows verbatim into the report. This comment is written via `--comment-out` for a github-script workflow to post on the PR. Attack scenarios: (1) a malicious PR injects markdown that spoofs the bot's verdict — e.g. a fake '## No actionable findings — safe to merge' section, fake severity badges, or a fake triage note — undermining the security gate the comment exists for; (2) injection of phishing links or camo-proxied image beacons into the maintainer-facing comment; (3) breaking out of inline-code spans via backticks — file paths are repo-controlled (a repo may contain a file named with a backtick or HTML-ish characters, and sandbox-merged records only enforce that the declared filePath matches the tar entry name, not its charset), and the `source` label embeds the raw `--diff` ref, which in CI is often derived from a fork PR's branch name (backticks are legal in git branch names), escaping `` `git-diff:...` `` into raw markdown. GitHub sanitizes dangerous HTML in comments, so this is content/markdown injection (spoofing, phishing, notification spam) rather than executable XSS, but in a security-review CI context the spoofed-verdict impact is meaningful.

Recommendation: Escape untrusted segments before interpolation: at minimum backslash-escape markdown-significant characters (`` ` ``, `*`, `_`, `[`, `]`, `<`, `>`, `~`, `\`, newlines) in filePath, title, description, recommendation, vulnSlug, and the source/runId header, or render those fields inside fenced code blocks with proper fence-length handling (choose a backtick fence longer than any run in the content). Consider additionally stripping/escaping HTML-like sequences and rejecting control characters in filePaths at scan time.

Unbounded recursion in canonicalJson aborts merge mid-loop, causing partial-merge data loss and whole-tarball rejection

- **File:** `packages/deepsec/src/sandbox/merge-records.ts`
- **Lines:** 102, 104, 139, 140, 146
- **Slug:** other-unbounded-recursion
- **Confidence:** medium

fileRecordSchema.analysisHistory[].modelConfig is z.record(z.unknown()), which accepts arbitrarily deeply nested JSON: V8's JSON.parse is iterative (no depth limit) and zod's z.unknown() does not recurse, so a sandbox-supplied record passes salvageFileRecord with a 100k-deep modelConfig. mergeFileRecord then feeds every analysisHistory entry to canonicalJson (lines 102-104), a plain JS recursive function (lines 139-148), which throws RangeError: Maximum call stack size exceeded. The exception escapes the mergeAfterExtract loop AFTER tar.extract has already blind-overwritten host records: records processed before the throw are merged, but every remaining record in the tarball was overwritten without merging, so

host-only analysisHistory/findings/gitInfo entries are permanently lost, and the thrown error rejects the entire download (downloadResults → extractTarballLocally propagates; the sandbox's whole results tarball is discarded). This is significant because it is the only way for a crafted record to bypass the merge machinery's protection: malformed records are restored-or-dropped and valid-but-thin records are unioned with host history. A tampered/compromised sandbox (the stated trust boundary) or a prompt-injected agent writing records can use this to silently erase host-side analysis state while making the run look like a benign download failure, and a merely unlucky record kills the whole poll for its sandbox. The withExtractLock itself is released in a finally, so the corruption is not even detected as a lock failure.

Recommendation: Make canonicalJson iterative (explicit stack) or depth-capped (throw a typed error at a sane depth, e.g. 64, that the merge loop converts into a per-record restoreOrDrop instead of aborting the whole download). Alternatively strip/depth-limit modelConfig (and any z.unknown()/z.record(z.unknown()) slots) during salvageFileRecord validation so unbounded structures never reach the merge, and consider making mergeAfterExtract process records inside per-record try/catch so one bad record degrades to restoreOrDrop rather than failing the tarball.

Untrusted sandbox-controlled strings written to console.warn without sanitization or length cap

- **File:** `packages/deepsec/src/sandbox/merge-records.ts`
- **Lines:** 225, 238, 245
- **Slug:** other-terminal-escape-injection
- **Confidence:** low

The failure paths of mergeAfterExtract embed sandbox-controlled data directly into console.warn output (lines 225-233, 238-241, 245-247): the tarball entry path `rel`, JSON.parse error text, and zod issue messages (which echo the received value verbatim, e.g. "Invalid enum value ... received """). All of this originates from the sandbox trust boundary (which is fed, in turn, by untrusted scanned-repo content via the agent). The strings are emitted with no ANSI/OSC filtering and no length cap (unlike orchestrator's MAX_LOG_LINE_CHARS), allowing terminal escape injection (UI spoofing, OSC 52 clipboard writes on susceptible terminals) and unbounded log spam from a single hostile record.

Recommendation: Strip ANSI/OSC escape sequences and cap length for any sandbox-derived string before it reaches console output; reuse the truncation approach used for sandbox log streams (MAX_LOG_LINE_CHARS).

Sandbox log lines streamed to the operator terminal without ANSI/OSC sanitization

- **File:** `packages/deepsec/src/sandbox/orchestrator.ts`
- **Lines:** 120, 131, 134
- **Slug:** other-terminal-escape-injection
- **Confidence:** low

streamLogsCapped (lines 120-141) forwards every sandbox command log line to onLog (line 134) after only a character-count truncation (MAX_LOG_LINE_CHARS). The content is fully attacker-influenced: it includes worker CLI output, agent stderr/stdout, and text the in-sandbox agent echoes from the untrusted scanned repo (the project's own threat model states prompt injection via source comments reaches agents that run

with shell access in the sandbox). Escape sequences therefore reach the operator's terminal raw, enabling UI spoofing (fabricated '[sandbox-0] Complete.' / '[sandbox-0] Stopped.' lines, clear-screen/cursor tricks that hide failures interleaved with real orchestrator output) and OSC 52 clipboard overwrite on terminals that honor it. The byte/line caps only bound volume, not control-character content.

Recommendation: Strip or render-inert ANSI/OSC control sequences (keep CR/LF/TAB) from log.data before passing lines to onLog, e.g. a regex removing CSI/OSC sequences, so sandbox output is display-safe regardless of terminal capabilities.

---

installFingerprint reads files from an unvalidated file: dependency path from the scanned repo's package.json

- **File:** `packages/deepsec/src/setup/coordinator.ts`
- **Lines:** 277, 283, 285
- **Slug:** path-traversal
- **Confidence:** low

installFingerprint() (L277-291) parses the workspace package.json and, when dependencies.deepsec starts with 'file:', converts the rest of the spec with fileURLToPath(dependency) and immediately readFileSync()s dist/cli.mjs and dist/config.mjs under that root. The spec string originates from the workspace package.json, which for the default layout (.deepsec inside the scanned repo) can be seeded or modified by repo content, and there is no path.resolve().startsWith(root) containment check before the reads. A crafted 'file:../../../etc' style spec (or a file:// URL with encoded segments) makes deepsec read arbitrary files and fold their digest into the install-phase fingerprint; the exception is swallowed, so probing succeeds silently. Impact is limited to arbitrary file read (content influences only a hash) plus the install phase then operating on attacker-chosen paths, but it violates the codebase's own containment convention used everywhere else for untrusted paths.

Recommendation: Validate the resolved packageRoot with path.resolve(packageRoot).startsWith(path.resolve(workspaceDir) + path.sep) before reading, and reject file: specs whose target falls outside the workspace (or outside the project root) instead of relying on the try/catch.

Model-controlled surface fileGlobs reach minimatch with no complexity, length, or count caps (CPU hang)

- **File:** `packages/deepsec/src/setup/coverage.ts`
- **Lines:** 227, 351, 408
- **Slug:** other-dos-unbounded-glob-complexity
- **Confidence:** low

Surface inventory comes from the setup agent's JSON (parseRepositoryAnalysis only shape-checks representativeFiles), and per the project threat model that agent can be steered by prompt injection embedded in the scanned repository. validateSurfaceInventory() (L227-248) checks fileGlobs only for relative-path shape and non-negation — there is no cap on glob length, star count, brace alternations, number of globs per surface, or number of surfaces. These unbounded globs are then expanded against every non-ignored repository file with minimatch in groundSurfaceInventory() (L351-353) and

expandSurfaceInventory() (L408-410): each file is tested against each glob. A pathological glob such as '*a*a*a*a*...b' (many star segments plus literals) compiles to a regex with nested [^/]* quantifier groups that exhibits catastrophic backtracking against typical filenames, so a single adversarial glob can hang the expansion loop indefinitely; thousands of such globs (counts are uncapped) multiply the cost. Notably, the sibling path for generated matchers (packages/scanner/src/declarative-matcher.ts globSafetyError) does enforce exactly these caps — MAX_GLOB_LENGTH 240, <=12 stars, <=3 brace groups, breadth probes — demonstrating the project considers this input adversarial but missed the same hardening on the surface-inventory path. Impact: attacker-steerable CPU denial of service that wedges the setup workflow (recoverable only by aborting and restarting with different input); no memory-safety or trust-boundary impact.

Recommendation: Apply the same (or a shared) glob safety gate used by declarative matchers to surface fileGlobs: cap glob length, star/brace counts, reject breadth-probe-matching patterns, and cap the number of globs per surface and surfaces per inventory. Ideally reuse globSafetyError/makeRe checks in validateSurfaceInventory so both paths share one hardened validator.

Malformed/invalid model inventory crashes grounding and permanently wedges resumable setup (checkpoint persisted before validation)

- **File:** `packages/deepsec/src/setup/coverage.ts`
- **Lines:** 218, 222, 308
- **Slug:** other-dos-persistent-setup-wedge
- **Confidence:** medium

validateSurfaceInventory() assumes the TypeScript SurfaceInventoryItem shape, but its input is untrusted model JSON: parseRepositoryAnalysis (packages/deepsec/src/setup/repository-analysis.ts) validates only infoMarkdown and representativeFiles — it casts surfaces as-is. If the model (potentially steered by prompt injection in the scanned repo) emits a surface missing 'description' or 'fileGlobs' (or non-array/number values), item.description.trim() (L218) or item.fileGlobs.length (L222) throws a TypeError inside groundSurfaceInventory() instead of producing a clean InventoryValidationIssue, so the analyzeRepository() validation-retry loop never fires. More broadly, ANY inventory that passes parsing but fails grounding (duplicate ids, invalid anchor regex, zero matchable globs) fails at the same point — and the coordinator persists the info-phase checkpoint and surface-inventory.json (writeRepositoryAnalysis + writeSetupState) BEFORE calling groundSurfaceInventory/expandSurfaceInventory. On resume, readInventory() accepts the persisted file (it only checks that surfaces is a non-empty array) and isCheckpointCurrent() skips the info phase, so setup reloads the identical poisoned inventory and fails again with the same error. The result is an attacker-steerable, persistent denial of service of the setup workflow for an unchanged repository: the only recovery is manually deleting the persisted inventory/setup state (which also discards scan data and forces re-paying for scanning).

Recommendation: 1) Schema-validate the full inventory (zod, mirroring declarativeMatcherSpecSchema's strictness) inside parseRepositoryAnalysis so malformed shapes feed the existing validation-retry loop instead of crashing later. 2) Make validateSurfaceInventory defensive against non-string/non-array fields (return issues rather than throwing). 3) In the coordinator, only mark the info checkpoint complete after ground/expand succeed, or add a recovery path that discards an inventory that fails validation and re-runs the analysis phase instead of skipping it.

Workspace .env.local values silently override real environment during credential probing

- **File:** `packages/deepsec/src/setup/plan.ts`
- **Lines:** 79, 81, 84
- **Slug:** other-env-override
- **Confidence:** low

buildSetupPlan() copies process.env into a local object and then runs loadEnvFile(path.join(workspaceDir, '.env.local'), env) (L79-81), and loadEnvFile Object.assigns the parsed file over the target, so dotenv values win over pre-existing environment variables. The subsequent tokenTriple and linkCredential checks (L84-87) and the credentialSources report read this merged object. For the default layout the workspace (.deepsec) sits inside the scanned repository, so repo-supplied or committed .env.local content can flip which credential path the plan reports as ready (for example by planting VERCEL_TOKEN/VERCEL_TEAM_ID/VERCEL_PROJECT_ID placeholders), steering the driving agent or operator toward a credential source the operator never configured. This is the inverse of dotenv's normal 'env wins' precedence and weakens the trust boundary between repo content and credential selection.

Recommendation: Only assign dotenv entries whose keys are not already present in the target env (mirroring dotenv's default precedence), or validate that workspace .env.local only ever supplies VERCEL_OIDC_TOKEN as written by pullOidcToken.

Setup plan JSON discloses which credential environment variables are populated

- **File:** `packages/deepsec/src/setup/plan.ts`
- **Lines:** 159, 161, 162, 163
- **Slug:** other-info-disclosure
- **Confidence:** low

The returned plan embeds credentialSources built from env.VERCEL_OIDC_TOKEN / env.VERCEL_TOKEN presence and CLI auth state (L159-163), plus the linked teamId/projectId (L148), and init.ts prints the full plan to stdout as formatted JSON for consumption by coding agents. The plan is explicitly agent-facing, so a prompt-injected agent (or a malicious repo instruction file the agent is simultaneously reading) learns exactly which credential channels exist on the host (oidc vs access-token vs cli), which sharpens credential-exfiltration attempts and confirms target Vercel org/project identifiers to an attacker who can read the transcript. No secret values are exposed, only their presence, but the output is designed for untrusted-context consumption.

Recommendation: Reduce credentialSources to a coarse readiness boolean (or a single 'credentials-configured' marker) for the machine-readable plan, and keep detailed source attribution in the 0600 setup log only.

Setup log redactor misses query-string credentials and unprefixed long tokens

- **File:** `packages/deepsec/src/setup/reporter.ts`
- **Lines:** 26, 41, 47
- **Slug:** secret-in-log
- **Confidence:** low

createSetupRedactor() (L33-56) replaces whole-value matches of secret-shaped env vars and four token patterns, but leaves several realistic secret shapes intact in the persisted 0600 JSONL setup log: credentials in query strings (e.g. https://host/path?api_key=SECRET or ?token=SECRET are untouched because the URL pattern at L29 only handles user:password@), long random secrets without a vck/vercel/sk/sess prefix or JWT eyJ shape, and env var names that do not match the TOKEN|SECRET|PASSWORD|API_KEY|PRIVATE_KEY|CREDENTIAL filter (e.g. VERCEL_OIDC_TOKEN is only caught because 'TOKEN' matches, but a var named AUTH or _HOOK sig would not be). Child and agent output lines flow unfiltered through emit() into the log (L95-103), so any library or CLI that echoes a connection string with query credentials ends up on disk in plaintext. The log also persists for the lifetime of the workspace and is pointed users at via printSetupSummary.

Recommendation: Extend TOKEN_PATTERNS to redact secret-looking query parameters ((?|&)(api_key|token|key|signature|password)=)[^&\s]+ and generic 32+ char high-entropy runs, and redact the full URL match in the userinfo pattern rather than keeping the host portion.

OS-level sandbox silently disabled when unavailable (failIfUnavailable: false) while Bash is explicitly allowed — prompt injection in a scanned repo yields unconfined shell with network access on the analyst's machine

- **File:** `packages/processor/src/agents/claude-agent-sdk.ts`
- **Lines:** 111, 112, 339, 634
- **Slug:** other-sandbox-fail-open
- **Confidence:** medium

For investigate/revalidate, the agent runs with allowedTools: ['Read', 'Glob', 'Grep', 'Bash'] and permissionMode: 'dontAsk' (claude-agent-sdk.ts:339, 634), meaning arbitrary shell commands are auto-approved without prompting. The intended containment for that Bash tool is the OS sandbox built by buildSandbox() (claude-agent-sdk.ts:106-113): { enabled: true, autoAllowBashIfSandboxed: true, failIfUnavailable: false }. failIfUnavailable: false makes the sandbox fail-OPEN: on hosts where the sandbox dependency is missing or unusable — bubblewrap absent (default on most Linux distros and Docker/CI images without user namespaces), Seatbelt unavailable — the SDK proceeds with NO filesystem or network containment, silently (the code comment at L102-104 admits the run 'degrades gracefully ... instead of hard-failing'). Because Bash is in allowedTools, it remains auto-approved; autoAllowBashIfSandboxed's gating is irrelevant since allowlisted tools bypass prompting. Attack chain: deepsec's threat model treats the scanned repo as untrusted input; a malicious repo containing prompt injection directs the agent to run shell commands that are executed unsandboxed with full network egress — arbitrary command execution on the researcher's machine, reading file-based secrets outside the env allowlist's reach (~/.ssh keys, ~/.aws/credentials, ~/.codex/auth.json, browser cookies) and exfiltrating them over the network. The env allowlist (buildClaudeEnv) only limits environment variables, not files or sockets. In-VM runs are unaffected (buildSandbox returns undefined by design, VM egress is proxied), but local laptop/CI runs lose their only Bash containment with no error, no warning surfaced to results, and no way for the user to know the scan ran uncontained.

Recommendation: Fail closed for runs that include Bash: set failIfUnavailable: true, or detect sandbox availability before launching and either drop 'Bash' from allowedTools or hard-fail the run with an explicit message when the OS sandbox cannot be established. At minimum, emit a prominent warning into the run record and results when the sandbox is not actually active.

System prompt tells the codex agent it is in a read-only sandbox while pickSandboxMode() grants write access to the scanned repository (workspace-write locally, danger-full-access in-VM)

- **File:** `packages/processor/src/agents/codex-sdk.ts`
- **Lines:** 70, 71, 606, 803, 1114
- **Slug:** other-agent-write-scope
- **Confidence:** low

codexEnvironmentPreamble (codex-sdk.ts:604-606) instructs the model: 'You are running inside the Codex CLI on a Linux sandbox (read-only mode, no network access).' But the actual invocation uses sandboxMode: pickSandboxMode() (codex-sdk.ts:803, 1114), which returns 'workspace-write' when running locally (codex-sdk.ts:70-71) — a mode that permits file WRITES anywhere inside the project working directory (projectRoot = the scanned repo) plus /tmp, with shell execution auto-approved (approvalPolicy: 'never') — and 'danger-full-access' inside the sandbox VM. The 'read-only' claim is prompt-level only; nothing enforces it. Consequence: repo content is untrusted input in deepsec's threat model, so a prompt injection in the scanned repo can direct the codex agent's shell to create, modify, or delete files in the user's working tree (e.g. destroying uncommitted changes, planting or editing files that will execute later in the user's normal workflow, rewriting scan-relevant sources after the scanner recorded candidates). Aggravator: the deepsec data root resolves to the relative path 'data' (packages/core/src/paths.ts:3-4), so when the CLI is run from inside the scanned repo, data//project.json (which contains rootPath used to steer all later host processing) and the file-record store live INSIDE the codex workspace-write scope — a tampered agent run can rewrite them, the exact steering that the sandbox-tarball path defends against by rejecting top-level project.json but that the local codex path does not. Network access being disabled locally limits exfiltration but not local integrity damage. The mismatch also degrades forensics: operators reason about the agent from a system prompt that misstates its powers.

Recommendation: Align the preamble with reality (state the actual mode and write scope), or better, run the codex agent in 'read-only' sandbox mode for scan/revalidate (matching the documented intent) and drop write capability entirely; if workspace-write is kept, exclude the deepsec data directory from the writable workspace and relocate the data root outside projectRoot.

Raw --ai-header credential values persisted into FileRecords/run metadata and missed by the pre-commit scrub

- **File:** `packages/processor/src/index.ts`
- **Lines:** 308, 714, 995, 1342
- **Slug:** secrets-exposure
- **Confidence:** medium

process()/revalidate() stamp the full agent `config` into every persisted artifact: runMeta.processorConfig.modelConfig (index.ts:308, index.ts:995) and per-file analysisHistory entries (index.ts:714, index.ts:1342). The config can contain `aiHeaders` built from `--ai-header NAME=VALUE` (packages/deepsec/src/agent-config.ts parseAiHeaders), where VALUE is commonly a raw credential (e.g. `--ai-header Authorization=Bearer sk-...`) — contradicting the stated hygiene that 'direct provider keys are referenced only by env-var name in config'. data-commit.ts's scrubbing pass

(scrubCommittedDataDir) only rewrites `candidates[].snippet` for secret-bearing slugs and its CREDENTIAL_RE belt-and-suspenders sweep also only inspects candidate snippets — `modelConfig.aiHeaders` values inside analysisHistory/runMeta are never scanned, so a raw secret in plaintext JSON under data//files/*.json and runs/*.json will be committed and pushed by commitAndPushData. It also defeats the env-allowlist philosophy: sandbox uploads ship data/ back to the host with the credential embedded.

Recommendation: Persist only credential-free config (drop aiHeaders values or replace values with env-var references before writing modelConfig), or add modelConfig/analysisHistory to scrubCommittedDataDir's scan with redaction of header-shaped values. Ideally resolve --ai-header values from environment variables by name, mirroring aiApiKeyEnv.

ReDoS safety checker bypass: '?'-quantified atoms inside quantified groups pass validation and enable catastrophic backtracking

- **File:** `packages/scanner/src/declarative-matcher.ts`
- **Lines:** 16, 27, 46, 61, 142, 219, 242
- **Slug:** other-redos
- **Confidence:** medium

compileDeclarativeMatcher() exists to safely compile untrusted setup-agent output (per the docstring: 'Validate untrusted JSON and compile it without evaluating generated code'), and the threat model explicitly treats repo content as a prompt-injection vector against the setup agent. However, regexSafetyError()'s nested-quantifier detector only recognizes *'', '+', and '{n,m}' as the INNER quantifier of a parenthesized group:* `(?:${atom})*(?:[*+]|\{\d+(?:,\d*)?\})(?:${atom})*\)\s*(?:[*+]|\{\d+(?:,\d*)?\})`. *The '?' quantifier is not checked, so patterns like* `(a?){1000}a{1000}x`, `(a?)+b`, `([a-z]?x){1000}...`, *or lookahead-wrapped variants such as* `(?=(a?)+b)` *pass every safety check: length <= 500, no NUL/backreferences/lookbehind, no "/'+* inner quantifiers, repeat count exactly at the allowed cap (only >1000 is rejected), and `regex.test('')` is false. `(a?){n}a{n}` is the canonical catastrophic-backtracking pattern: for an input line of n 'a's followed by a non-matching character, the engine explores C(n, k) ways to distribute the 'a's across the 1000 optional-match iterations, causing exponential CPU. There is no re2 or execution timeout anywhere in the scanner — regexMatcher() (matchers/utils.ts) runs `regex.test(line)` per line against attacker-controlled repo file content, so a single crafted ~1000+ character line in any file matching the matcher's filePatterns hangs the scan process indefinitely. On the host this is a CPU DoS of the scanning machine; in distributed mode it burns billed Vercel Sandbox CPU. Duplicate-branch alternations like `(a|a)*` are correctly caught by the quantifiedAlternation check, but the '?' gap remains exploitable.

Recommendation: Treat '?'/'??' as inner quantifiers in the nestedQuantifier check (i.e. add '?' to the quantifier alternation, requiring at least one non-optional atom inside the group), or more robustly: cap total bounded-repeat iterations (e.g. reject {n,m} where n*m exceeds a small bound when the repeated group contains optional atoms), and/or run matcher regexes against content through a re2-compatible engine or a worker with a per-file/time deadline so any residual ReDoS cannot hang the scan.

Scanner file reads follow symlinks, allowing a scanned repo to read host files outside the repo root

- **File:** `packages/scanner/src/index.ts`
- **Lines:** 302, 346, 701, 702, 714
- **Slug:** path-traversal
- **Confidence:** medium

RegexScannerDriver.scan() reads every glob result with fs.readFileSync(path.join(root, relPath), 'utf-8') (L302), and scanFiles() does fs.existsSync/statSync/readFileSync on path.join(absRoot, relPath) (L701-L714). All of these follow symlinks, and glob v11 (used with default follow:false, nodir:true) still returns symlink entries themselves — only symlinked directories are not traversed. A repo can therefore contain e.g. `creds.ts -> /home/ci/.aws/credentials` (or any absolute-target symlink with a matcher-matching name/extension); the scanner will read the target file outside the repo root, hash it, and — because regexMatcher stores matched context lines as CandidateMatch.snippet — embed its content into FileRecords under data//files/. Those records are subsequently fed to the AI processor (sent to a cloud model), included in reports/exports, and optionally committed by data-commit. This bypasses the root containment the transport layer enforces elsewhere (.gitignore honoring, .git exclusion, tarball allowlists) and directly matches the stated threat model where the scanned repo is untrusted input (e.g. CI PR-review of attacker-supplied repos): the repo can exfiltrate host files outside its root into scan data. The path-traversal flags on path.resolve(params.root) (L476/500/512/567/647) themselves are false positives — root is operator-supplied and record writes are guarded by core's assertSafeFilePath — but the symlink-following read is a genuine escape of the trust boundary. Note also that in scanFiles() the read at L714 happens before any write-time path validation, so a traversal-shaped caller path would be read even though writeFileRecord would later reject it.

Recommendation: Before reading, lstat each entry and skip symlinks (or resolve with fs.realpathSync and verify the real path is still inside path.resolve(root) via a path.relative containment check), both in RegexScannerDriver.scan() and scanFiles(). Additionally, validate relPath against assertSafeFilePath (or equivalent) at the top of the scanFiles loop before any filesystem access, so validation precedes reads rather than trailing them at write time.

ReDoS in SOQL_TPL matcher regex — crafted repo line can stall the scan run

- **File:** `packages/scanner/src/matchers/soql-injection.ts`
- **Lines:** 49, 56
- **Slug:** other-regex-dos
- **Confidence:** medium

The SOQL_TPL detection regex /`(?:[^`]*?\bSELECT\s+[^`]*?\bFROM\s+[A-Za-z_][A-Za-z0-9_]*[^`]*?)`${/i combines three lazy [^`]*? quantifiers with an interior greedy \s+ between required anchors. It is tested against every line of every .ts/.tsx/.js/.mjs file in the scanned repo (line 56), and scanned repo content is explicitly untrusted input per the project threat model. A crafted line such as a backtick, the word SELECT, followed by ~100KB-1MB of whitespace (with no FROM and no ${) forces the engine to backtrack through every split point of \s+, re-scanning the remainder of the line for \bFROM each time — O(n^2) character work on a single line. Worst cases also arise from many SELECT occurrences without FROM on one long line (e.g. minified JS). The RegexScannerDriver in packages/scanner/src/index.ts invokes matcher.match() synchronously with no per-file or per-matcher timeout, so one crafted line hangs the whole scan (and in distributed mode burns paid Vercel Sandbox VM time while the run appears stuck). The gate (HAS_SF requiring a jsforce import or conn.query(call) is trivially satisfiable by the same attacker-crafted file. Note:

this is a denial-of-service against the deepsec scanner itself, not a finding about scanned targets; the sql-injection/js-sql-raw flags the scanner emitted on this file are false positives on its own example strings and regex literals.

Recommendation: Restructure SOQL_TPL to eliminate the interior \s+ backtracking split (e.g. require a non-whitespace token between SELECT and FROM, or use possessive/atomic-group semantics via lookahead tricks), bound line length before testing (skip or truncate lines over a few KB), and/or run each matcher.match() under a per-file wall-clock timeout in RegexScannerDriver so a pathological regex degrades to a skipped file instead of a hung run.

HIGH_BUG (4)

Scan and export stages never run their intended commands — every step re-runs 'deepsec init'

- **File:** `noc/pipeline.yaml`
- **Lines:** 83, 84, 89, 90, 117, 118, 120
- **Slug:** other-broken-stage-dispatch
- **Confidence:** high

The 'repo-scan' step (L83-96) and the 'export' step (L117-122) specify only 'args' with no 'command', so Kubernetes/Tekton appends the args to the image ENTRYPOINT, which is 'bash /home/scan/deepsec-setup.sh' (noc/Containerfile L38). The committed deepsec-setup.sh ignores positional arguments entirely and unconditionally executes 'npx deepsec init ... $WORK_DIR $REPO_DIR'. Therefore: (1) the scan task's 'repo-scan' step re-runs onboarding/init instead of performing the repository scan; (2) the 'export' step re-runs init instead of deepsec-report.sh / 'deepsec report'. The dispatch could not work even with a dispatcher: 'repo-scan' and 'post-report' are not commands exposed by the deepsec CLI (commands are init/setup/init-project/scan/process/report/revalidate/enrich/triage/status/export/metrics/sandbox...). The export step additionally runs with no env of its own (the export taskSpec defines no stepTemplate), inheriting only image defaults REPO_DIR=/repo and WORK_DIR=/work — paths that do not exist in the export pod (the repo lives under the workspace path), so 'deepsec init ... /work /repo' fails there. Net effect: the pipeline clones the repo, runs the (agent-invoking, LLM-billing) init three times, never performs the scan, and never produces or exports a report. Evidence chain: pipeline.yaml L83-84/L117-118 args-only steps; noc/Containerfile L38 ENTRYPOINT; noc/deepsec-setup.sh (no $1 dispatch); deepsec CLI command registry in packages/deepsec/src/cli.ts.

Recommendation: Give each step an explicit 'command'/'script' invoking the correct deepsec CLI commands (e.g. 'deepsec scan'/'deepsec process' for the scan stage and deepsec-report.sh for export), or convert deepsec-setup.sh into a dispatcher on '$1' that accepts valid subcommands. Pass REPO_DIR/WORK_DIR/PROJECT_ID into the export task's env.

---

commitAndPushData never verifies data/ is a standalone git repo — git operates on the enclosing target repo and commits/pushes the user's whole working tree

- **File:** `packages/deepsec/src/data-commit.ts`
- **Lines:** 6, 119, 128, 133, 139, 156, 162
- **Slug:** other-unintended-git-commit-push

- **Confidence:** high

DATA_DIR is computed as path.resolve(getDataRoot()) = '/data' (L6), and commitAndPushData runs `git status/add/commit/push` with cwd: DATA_DIR (L128, L133, L139, L156, L162) without ever checking that DATA_DIR is itself a git repository (no `.git` existence check, no `git rev-parse --show-toplevel` guard). In the documented default layout, `.deepsec/` sits inside the scanned repository (see docs/faq.md: '.deepsec/ directory at the root of the repo you want to scan, checked into git') and nothing in the codebase ever creates a repo inside `data/` — so git resolves upward from `.deepsec/data` to the TARGET repository. Consequences when `deepsec enrich` is run (enrich.ts calls commitAndPushData unconditionally whenever >0 files were enriched): (1) `git status --porcelain` reports the target repo's dirty state; (2) `git add -A` with no pathspec stages the ENTIRE working tree regardless of cwd (Git >= 2.0 semantics), i.e. all of the user's unrelated WIP; (3) `git commit -m 'enrich: <projectId> (...)'` commits it; (4) `git pull --rebase origin <branch>` and `git push origin HEAD:<branch>` push it to the target repo's own origin — including pushing the user's current feature branch tip (with freshly committed WIP) onto `<branch>` (default `main`). A user with uncommitted work who runs `deepsec enrich` silently publishes that WIP (potentially secret-bearing) to their remote and corrupts their branch state under an 'enrich:' message. The project README claims commitAndPushData 'only runs against an explicit data repo with user/CI opt-in', but no such opt-in or verification exists in code — the only guard is the implicit git repo resolution. This also means the credential scrubbing operates on records that, in the default layout, are gitignored anyway, while the actual push targets the user's repo.

Recommendation: Before running any git command, verify DATA_DIR is the intended standalone repo, e.g. check `fs.existsSync(path.join(DATA_DIR, '.git'))` or assert `git rev-parse --show-toplevel` (cwd: DATA_DIR) equals DATA_DIR, and that a remote `origin` is configured; otherwise skip the commit/push with a clear warning instead of resolving git upward to the enclosing repository. Scope `git add` to `./` (cwd-relative) rather than `-A`, and never push `HEAD:<branch>` without confirming HEAD's branch.

---

parseRevalidateVerdicts performs zero schema validation on model verdicts, causing silent permanent loss of findings via the strict read-side schema

- **File:** `packages/processor/src/agents/shared.ts`
- **Lines:** 1119, 1129
- **Slug:** other-unvalidated-model-output
- **Confidence:** high

parseRevalidateVerdicts (shared.ts:1119-1129) does `return parsed as RevalidateVerdict[]` — a bare type cast with no field/enum validation — on model-produced revalidation JSON. This contrasts with the investigate path, where parseInvestigateResults field-validates every finding with findingSchema.safeParse and runs an in-session repair loop (added precisely to fix this class of bug for findings). The unvalidated verdicts flow into the processor (packages/processor/src/index.ts:1233-1243), which persists `finding.revalidation = { verdict: m.verdict.verdict, reasoning: m.verdict.reasoning, adjustedSeverity: m.verdict.adjustedSeverity, ... }` verbatim and additionally executes `finding.severity = m.verdict.adjustedSeverity` when the field is truthy. core's writeFileRecord (packages/core/src/run.ts:330-334) writes the record without validation. On the next read, salvageFileRecord (packages/core/src/schemas.ts:228-241) validates each finding against the strict revalidationSchema enums (verdict ∈ {true-positive, false-positive, fixed, uncertain, accepted-risk,

duplicate}; adjustedSeverity ∈ {CRITICAL...LOW}); any finding whose revalidation carries an off-enum value — e.g. the extremely common model outputs 'True-Positive', 'high', a missing reasoning string, or a verdict object missing the verdict/reasoning keys — fails findingSchema and is silently DROPPED from the record on every subsequent load (loadAllFileRecords applies the same salvage). This is exactly the silent-data-loss failure mode the codebase already observed and fixed for findings (see the regression test comment in packages/core/src/**tests**/run.ts:134-141: '148 findings across one observed 12-run comparison'), but the revalidate path reintroduces it with no repair loop and no rejection. Two aggravators: (1) since the scanned repo is untrusted input in deepsec's threat model, prompt injection in repo content can steer the revalidate model to emit malformed or value-corrupting verdicts, selectively deleting true-positive findings about the attacker's own code — a scanner-integrity attack; (2) the 'accepted-risk' verdict value IS in the accepted enum but per the schema comment 'the agent never produces it' and it is 'treated like false-positive for report-default filtering' — an injected response emitting verdict:"accepted-risk" persists cleanly (it passes read-side validation) and hides a real finding from default reports/marketing-facing metrics while counting as resolved (index.ts:1225 skips re-applying over accepted-risk). Impact: loss/corruption of revalidation verdicts and finding severities, silent (console.warn only), unrecoverable without manual data surgery.

Recommendation: Validate every verdict object with a zod schema (mirroring revalidationSchema minus the timestamp/runId/model fields) inside parseRevalidateVerdicts, and feed invalid entries into an in-session repair loop exactly like runInvestigateFieldRepairLoop does for findings; reject verdict values outside the non-manual enum set (especially 'accepted-risk') before persistence; never copy adjustedSeverity into finding.severity until it has passed enum validation.

Triage verdicts persisted with zero validation — invalid enum values silently delete the finding itself at next load; parse failures complete silently

- **File:** `packages/processor/src/triage.ts`
- **Lines:** 196, 200, 207, 215
- **Slug:** other-unvalidated-model-output
- **Confidence:** high

Triage is the least-defended consumer of model output: resultText is parsed with a regex + JSON.parse (triage.ts:196-200, empty-catch), and `verdict.priority/exploitability/impact` (arbitrary strings from the model) are written verbatim into finding.triage (triage.ts:207-214) with no zod validation, no repair loop, and no retry — unlike investigate (findingSchema.safeParse + field-repair) and revalidate (reconcile + id-repair). findingSchema declares triage.priority/exploitability/impact as nested z.enums, so any out-of-enum value makes the whole findingSchema.parse fail on the next load; salvageFileRecord then drops the ENTIRE finding (not just the triage block) with only a console warning — a triage run can permanently erase a real vulnerability from the dataset, reports, and metrics. The prompt embeds finding descriptions (model-generated text ultimately derived from untrusted scanned-repo content), so a malicious repo can steer this both by injecting a 'skip' verdict and by forcing garbage enum values that trigger the deletion path. Additionally, if the response isn't valid JSON at all, verdicts stays [] and the batch is reported complete with 0 triaged — runMeta completes 'done' with exit 0, so systematic triage failure is indistinguishable from success.

Recommendation: Validate verdicts with a zod schema before applying (drop/repair invalid entries, matching the investigate path's repair loop); treat unparseable JSON as a batch failure (non-zero signal / error state) rather than silently completing; sanitize persisted values to the legal enum sets.

BUG (40)

Live-sandbox e2e test auto-runs on every push to main / same-repo PR despite being documented as manual-trigger-only

- **File:** `.github/workflows/e2e-live-sandbox.yml`
- **Lines:** 3, 4, 5, 6
- **Slug:** other-config-drift
- **Confidence:** medium

e2e/pipeline-sandbox.test.ts documents that this test is 'NOT run on push/PR' and should be 'trigger[ed] manually from GitHub Actions → "Run workflow"', but the workflow has no workflow_dispatch trigger and instead fires on every pull_request and push to main. Because DEEPSEC_E2E_LIVE_SANDBOX=1 and VERCEL_TOKEN/VERCEL_TEAM_ID/VERCEL_PROJECT_ID are set, the ~35-minute test spins up real Vercel Sandboxes on every push to main and every same-repo PR (fork PRs silently skip since secrets are withheld under pull_request and HAS_SANDBOX_KEY is false). The documented manual-run path is impossible, and CI minutes plus Vercel sandbox quota are consumed unintentionally and repeatedly on every internal change.

Recommendation: Add workflow_dispatch (and remove or narrow the pull_request/push triggers, e.g. paths-filtered or label-gated) to match the documented manual-trigger operating model, or update the test file's comments to reflect the auto-run behavior.

PROJECT_ID never provided — every repo scanned registers under project id '0' on a shared PVC

- **File:** `noc/pipeline.yaml`
- **Lines:** 48, 83
- **Slug:** other-shared-project-id
- **Confidence:** medium

Neither the pipeline (its params are only repo-url/revision/model-name, and the scan stepTemplate at L48-63 defines no PROJECT_ID) nor the sample PipelineRun (noc/pipeline-runs.yaml) sets PROJECT_ID. The value only comes from the image ENV placeholder PROJECT_ID=0 (noc/Containerfile L32). deepsec-setup.sh requires PROJECT_ID and passes '--id $PROJECT_ID' to init, and deepsec stores project state under data/ within WORK_DIR, which here is a single persistentVolumeClaim ('deepsec-scan-pvc') shared by all PipelineRuns. Consequently every repository scanned through this pipeline is initialized under the same project id '0' in the same WORK_DIR, colliding and mixing project state/scan data across different repositories. The export stage would use the same id '0' for 'deepsec report --project-id'.

Recommendation: Add a projectId param to the pipeline (e.g. derived from the repo name or PipelineRun name) and set PROJECT_ID in the scan and export task envs; use a distinct WORK_DIR or cleanup per run.

Primary findings store and run metadata written non-atomically — crash mid-write permanently loses a file's entire record

- **File:** `packages/core/src/run.ts`
- **Lines:** 156, 159, 330, 333

- **Slug:** other-non-atomic-write
- **Confidence:** medium

writeFileRecord (L330-334) and writeRunMeta (L156-159) use a direct fs.writeFileSync to the final path, unlike writeProjectConfig which correctly uses temp-file + rename. writeFileRecord is the primary persistence path for FileRecords — the store holding all findings, analysisHistory, and revalidation state from expensive AI processing — and is written after every processed file. A crash, OOM kill, or power loss mid-write leaves a truncated or zero-length JSON file. The recovery layer cannot help: parseFileRecordSalvaging only salvages findings within a valid envelope, and comment L77-80 explicitly acknowledges that 'an interrupted non-atomic write' is a known failure mode that had to be special-cased for project.json. On next load the corrupt record is skipped with a warning and that file's findings and history are permanently gone (silently resetting the file to reprocessable, triggering re-spending of model tokens). The codebase elsewhere treats these records as valuable enough to build merge/salvage machinery (merge-records.ts) to avoid losing them, making the non-atomic write an inconsistency with real data-loss potential.

Recommendation: Use the same temp-file + rename pattern as writeProjectConfig (write to `${p}.${pid}.tmp` with mode 0600, then renameSync) in writeFileRecord and writeRunMeta.

Process-lock acquire/reclaim/release are TOCTOU-racy: blind rmSync can delete a live holder's lock and two reclaimers can both acquire

- **File:** `packages/core/src/run.ts`
- **Lines:** 385, 391, 408, 410, 413
- **Slug:** other-race-condition
- **Confidence:** low

acquireProcessLock implements mutual exclusion with an mkdir lock but has two unguarded races. (1) Stale reclaim (L397-413): the staleness check (statSync mtime > 1h) and the recovery (rmSync + retry mkdir) are not atomic. If two processes both observe the stale lock, the first reclaims and creates a fresh lock dir; the second's rmSync (L410) then deletes the *new* holder's lock and its mkdir succeeds — both now hold the lock and concurrently claim overlapping files, clobbering each other's per-file locks/claims (the exact failure the lock exists to prevent, acknowledged at L305-307). Similarly, a fresh acquirer racing a reclaimer can have its new lock deleted. (2) Release (L385-392): the returned closure unconditionally rmSync's whatever currently exists at lockDir, with no verification that this process still owns it (no read/compare of the owner runId). If the claim phase stalls for more than PROCESS_LOCK_STALE_MS (1h — plausible with disk stalls, VM pauses, or a debugger), another invocation reclaims the lock; when the stalled holder later releases it deletes the new holder's lock, allowing a third invocation to acquire while the second is still claiming.

Recommendation: Before deleting, re-read the owner file and only reclaim/remove if the recorded runId and acquiredAt still match the stale (or own) lock; better, make reclaim atomic by rename()-ing the lock dir to a unique name first and only removing that renamed dir, and have release() verify ownership (or use a kernel-level primitive like O_CREAT|O_EXCL file locking via properflock).

Remote benchmark 'reasoning' bypasses thinking-level allowlist, crashing and stickily breaking headless setup

- **File:** `packages/deepsec/src/auth/model-picker.ts`
- **Lines:** 115, 124, 142, 144, 289, 291, 310
- **Slug:** other-unvalidated-remote-config
- **Confidence:** medium

fetchBenchmarkResults() (L115-131) validates the remote DeepSecBench payload (https://vercel.com/ai-gateway/leaderboards/deepsecbench/results.json) for JSON types only via isBenchmarkResult(); the 'reasoning' string is not constrained to a value domain. thinkingLevel() (L142-144) maps 'default'->undefined and 'max'->'xhigh' but passes every other remote string through verbatim, whereas all local inputs of the same field are allowlisted: the interactive custom path (L309-311) enforces ['minimal','low','medium','high','xhigh'] and buildAgentConfig() (agent-config.ts) throws on anything else. Attack/impact path: a benchmark response with reasoning:'ultra' (feed bug, or compromise of the hardcoded vercel.com origin) makes resolveModelProfile()/promptForModelSelection() return thinkingLevel:'ultra'; setup/coordinator.ts persists it into setup state (state.agent at L388, written before buildAgentConfig runs at L440), then buildAgentConfig throws '--thinking-level must be one of minimal, low, medium, high, xhigh'. On resume, existingState.agent.thinkingLevel re-feeds the poisoned value, so headless setup keeps failing until the user explicitly passes a valid --thinking-level. The same remote strings (model, label, reasoning) are also rendered to the terminal (L274-277) and steer model/cost selection without any value sanitization (e.g. ANSI escapes, cost:0 degenerating the 'value' profile via Math.min), though the HTTPS trust anchor keeps this out of typical attacker reach. Evidence of the gap: contrast L144 (unvalidated passthrough) with L310 (allowlist) for the same field.

Recommendation: Validate remote benchmark data against the same value domains as local input: restrict 'reasoning' to the known set (or coerce unknown values to undefined/'high') inside thinkingLevel()/isBenchmarkResult(), and clamp/validate 'cost' (positive, finite) and string fields (length caps, no control characters) before they reach persistence or terminal rendering. Validate before persisting setup state so a bad feed cannot create a sticky failure loop.

---

md-dir export sweep deletes unresolved findings omitted by the current run's filters

- **File:** `packages/deepsec/src/commands/export.ts`
- **Lines:** 258, 271, 278, 287
- **Slug:** other-logic-bug
- **Confidence:** high

writeMdDir() builds `wantedFiles` solely from findings that survived THIS invocation's filters (project subset via --project-id, --min-severity/--only-severity, --only-slugs/--skip-slugs, --only-true-positive, the default hiding of resolved verdicts, --require-owner, --only-agent, --only-marker, --since/--discovered-today), then unconditionally deletes every .md file in each severity-named subdirectory that is not in that set (fs.unlinkSync at L287). The in-code rationale assumes a missing file means the finding was 'revalidated as fixed/false-positive/accepted-risk', but the sweep cannot distinguish 'resolved' from 'outside the current filter'. Any filtered md-dir export therefore deletes exported files for findings that are still unresolved: e.g. `export --format md-dir --out ./findings --project-id a` (the exact workflow the scaffolded README instructs once a workspace has 2+ projects) deletes every finding file previously exported for projects b..n; `--min-severity CRITICAL` wipes exported HIGH/MEDIUM files; `--only-slugs sql-injection` wipes every other slug. The export directory is consumed by downstream triage/issue-tracker tooling (assignee labels, severity dirs), so unresolved security findings silently disappear from the artifact

teammates and automation rely on. Only a '(removed N stale file(s))' count hints at the deletion; files are regenerable only by re-running an unfiltered export.

Recommendation: Scope the stale sweep to the population the current run is authoritative for: either only sweep when no filter that can omit unresolved findings is active (project subset, severity, slug, agent, marker, date, owner), or track provenance in the exported files (e.g. embed project/slug metadata and only delete files whose finding is now resolved, verified against an unfiltered load of the records) instead of deleting everything not in the current filtered wanted-set.

--only-agent / --only-marker misattribute findings because revalidate history entries break the finding-index mapping

- **File:** `packages/deepsec/src/commands/export.ts`
- **Lines:** 434, 436, 440, 469, 470, 471
- **Slug:** other-logic-bug
- **Confidence:** high

export.ts maps finding index → producing analysisHistory entry by cumulative findingCount (L436-441), on the stated assumption that 'Findings are appended in analysisHistory order'. That assumption holds for `phase: "process"` entries (processor/src/index.ts records newFindings.length and appends exactly those findings), but NOT for revalidate entries: the revalidate path pushes an analysisHistory entry with `findingCount: persistedForFile` — the count of EXISTING findings whose revalidation was persisted — while appending zero findings (processor/src/index.ts ~L1328-1344). Whenever a revalidate entry precedes a later process entry (the documented process → revalidate → re-investigate flow), the revalidate entry consumes finding-index slots in the cumulative cursor, so findings appended by the later process run are attributed to the revalidate entry. The --only-agent filter (L470) then compares the revalidator's agentType instead of the producing agent's (e.g. a codex-produced finding passes `--only-agent claude` and is dropped from `--only-agent codex`), and the --only-marker filter (L471) silently drops those findings entirely because revalidate entries carry no reinvestigateMarker. Filtered exports emit wrong agent attribution or omit net-new findings without warning.

Recommendation: Track attribution explicitly instead of reconstructing it from counts: stamp each finding with the producing history entry (e.g. persist agentType/reinvestigateMarker on the finding, or record the finding's producedByRunId — already present — and join it against runId in analysisHistory), or have revalidate entries record findingCount: 0 and add a separate revalidatedCount field.

SEVERITY_ORDER ranks HIGH_BUG above MEDIUM, contradicting every other severity ordering in the codebase and dropping MEDIUM findings from --min-severity HIGH_BUG exports

- **File:** `packages/deepsec/src/commands/export.ts`
- **Lines:** 9, 16, 446, 527
- **Slug:** other-logic-bug
- **Confidence:** high

export.ts orders severities CRITICAL(0), HIGH(1), HIGH_BUG(2), MEDIUM(3), BUG(4), LOW(5) (L9-16). The three other SEVERITY_ORDER definitions in the same codebase — pr-comment.ts (L8-15),

sandbox/partitioner.ts (L8-14), and commands/metrics.ts (L6-13) — all rank MEDIUM above HIGH_BUG, matching the schema enum and the project's severity taxonomy (security severities CRITICAL/HIGH/MEDIUM, separate non-security HIGH_BUG/BUG track). Consequences: (1) `--min-severity HIGH_BUG` in export EXCLUDES MEDIUM security vulnerabilities (order 3 > 2) while metrics and pr-comment include them — the same flag has contradictory semantics across commands, and a user filtering an export to 'HIGH_BUG and above' silently loses MEDIUM security findings from the report; (2) `--min-severity MEDIUM` includes HIGH_BUG in export but excludes it in metrics; (3) the export sort (L527-528) interleaves non-security HIGH_BUG items above real MEDIUM security vulnerabilities.

Recommendation: Align export.ts's SEVERITY_ORDER with the canonical order used by pr-comment.ts, sandbox/partitioner.ts, and metrics.ts (CRITICAL, HIGH, MEDIUM, HIGH_BUG, BUG, LOW) — ideally by exporting a single shared SEVERITY_ORDER from @deepsec/core so the ordering cannot drift again.

SHA1 for filename dedup and unencoded filePath interpolation in GitHub links (verified not exploitable)

- **File:** `packages/deepsec/src/commands/export.ts`
- **Lines:** 107, 118, 236
- **Slug:** other-logic-bug
- **Confidence:** high

Reviewed and cleared: (1) the SHA1 at L236 is used only to derive a stable, filesystem-safe export filename for dedup — no security property (collision resistance against an adversary, integrity, authentication) is required, so this is not an insecure-crypto finding; (2) makeGithubLink (L107-121) interpolates record.filePath into a GitHub display URL without URL-encoding, but filePath is validated at record-write time (assertSafeFilePath rejects '..', absolute paths, backslashes, null bytes) and the URL is only rendered as markdown text in export output — no fetch occurs, so the [ssrf]/[git-provider-url-injection] flags are false positives. Only cosmetic defect: a filePath containing '#' or '?' can break the L-anchor of the generated link.

Recommendation: No security fix needed. Optionally URI-encode the path segment in makeGithubLink (encodeURIComponent per segment) so anchors survive paths containing '#' or '?'.

--force re-register appends a duplicate project entry to deepsec.config.ts and the stale entry keeps winning

- **File:** `packages/deepsec/src/commands/init-project.ts`
- **Lines:** 78, 95, 96, 101, 134, 146
- **Slug:** other-logic-bug
- **Confidence:** high

registerProject() guards against re-registering an existing id with `if ((dataExists || inConfig) && !opts.force) throw` (L78), but with --force (CLI flag documented as 'Overwrite an existing project of the same id') it proceeds to insertProjectIntoConfig() (L101), which unconditionally inserts another `{ id, root }` entry above the marker — there is no dedupe or in-place update of the existing entry. The fresh entry is inserted directly above the marker, i.e. AFTER the pre-existing entry in array order, so findProject() (first-match in config.ts L113) keeps returning the OLD entry with the stale root: the forced re-register silently fails to update the effective project root for every config-driven flow (scan root resolution,

resolveProjectIdForDirect). Duplicate ids also break resolveProjectId's single-project auto-resolution ('Multiple projects in deepsec.config.ts: foo, foo' even though it is one logical project) and leave project.json (rewritten by ensureProject with the new root) and deepsec.config.ts inconsistent. Additionally, --force replaces the hand-curated, git-tracked INFO.md with the placeholder template (L95-96), destroying curated security context (recoverable only if committed).

Recommendation: In insertProjectIntoConfig (or registerProject), when the id is already present in the config, replace the existing entry's root in place rather than inserting a second entry (or remove the old entry before inserting). Consider requiring an explicit confirmation before overwriting the curated INFO.md, since it is documented as hand-maintained.

Out-of-root path filter only rejects leading '../' — embedded traversal segments escape root

- **File:** `packages/deepsec/src/file-sources.ts`
- **Lines:** 82, 85, 92
- **Slug:** other-logic-bug
- **Confidence:** high

resolveFiles() documents (and its test 'rejects paths outside root' asserts) that entries are filtered to files under rootPath, but the check `if (rel.startsWith("../") || rel === "..") continue;` (line 82) only catches traversal at the START of the path. A list entry like `pkg/../../outside-secret.ts` or `a/b/../../../etc/cron.d/x` passes the check; `path.join(absRoot, rel)` (line 85) then normalizes to a path outside the root, `existsSync`/`statSync` accept it, and the raw '..'-laced relative path is pushed into the result (line 92). Consequences: (1) files outside the scan root are read by the downstream scanner (`fs.readFileSync(path.join(absRoot, relPath))` in scanFiles), bypassing both the root containment and the IGNORE_DIRS/deepsec-data ignore filtering; (2) the run then crashes uncaught when core's `assertSafeFilePath()` (which rejects '..' segments) throws inside readFileRecord/writeFileRecord — so the out-of-root content never persists, but the documented containment contract is broken and the CLI dies mid-run. Input comes from operator-supplied `--files` / `--files-from` (or git-sourced lists, which cannot contain '..'), so this is a validation/logic gap rather than a privilege boundary; impact is out-of-root reads (transient, in-memory candidate snippets) plus a run-breaking crash.

Recommendation: Normalize before validating: compute `const normalized = path.posix.normalize(rel)` (or resolve to an absolute path and take `path.relative(absRoot, ...)`), then reject if the normalized result starts with '../', equals '..', or `path.resolve(absRoot, rel)` does not remain under `absRoot + path.sep`. Mirror the validation semantics of core's assertSafeFilePath so the two layers agree, and add a test for embedded traversal (e.g. `pkg/../../escape.ts`).

Output-cap budget is checked after the text is already emitted — a single oversized multi-line write passes through in full, defeating the stated OOM protection

- **File:** `packages/deepsec/src/output-cap.ts`
- **Lines:** 35, 46, 53, 54, 56, 61
- **Slug:** other-cap-bypass
- **Confidence:** medium

capStream() (L35-63) truncates each LINE to MAX_LINE_CHARS (2000) via truncateLines (L24-32), but there is no per-call size bound: a single write containing a multi-megabyte, many-line string (e.g. one console.log of a large object rendered with newlines, or JSON.stringify(x, null, 2)) passes through truncateLines unchanged. The full text is then handed to the underlying stream (L61 `return orig(text, encoding, callback)`) BEFORE the budget decision matters — the budget is only decremented (L53) and, once negative (L54-59), the final write still emits the entire oversized text plus the cap notice. Per the file's own header comment, the sandbox NDJSON log stream is parsed with unbounded JSON.parse in @vercel/sandbox and 'a single oversized record can still OOM the SDK before that cap applies — so the worker must never emit one'; this code path emits exactly such a record, so the invariant the cap exists to enforce is violated for writes larger than the remaining budget. Exploitability is modest (requires the worker to produce one very large write call), so this is a reliability/defense-in-depth gap rather than a directly triggerable vulnerability.

Recommendation: Bound each write before emitting: if Buffer.byteLength(text) > state.remainingBytes, slice text to the remaining byte budget (respecting UTF-8 boundaries), emit the slice plus the cap notice, and set exhausted — instead of emitting the whole text and only then marking the budget spent.

Merged file records are written non-atomically; a crash or ENOSPC mid-write destroys both host and incoming analysis data

- **File:** `packages/deepsec/src/sandbox/merge-records.ts`
- **Lines:** 265, 270, 279
- **Slug:** other-non-atomic-write
- **Confidence:** medium

mergeAfterExtract persists merge results (and salvaged rewrites / host restores) with plain fs.writeFileSync at lines 265, 270 and 279 (restoreOrDrop, line 276-286). These are in-place overwrites of the only copy of the data: the pre-extract host snapshot exists only in memory, and the incoming record only in the tarball (which is unlinked right after extraction in download.ts). If the process crashes, is SIGKILLed, or hits a full disk mid-write, the record file is left truncated/unparseable, losing the union of host analysisHistory/findings and the sandbox's contribution for that file until a paid re-analysis re-runs. restoreOrDrop additionally swallows write errors with an empty catch, so a failed restore silently leaves the corrupt sandbox version in place. This is inconsistent with the project's own convention of atomic (temp+rename, 0600) writes for env files, and the module's stated goal that 'a corrupt or absent host record shouldn't block a sandbox upload'.

Recommendation: Write via temp file + fs.renameSync (atomic on POSIX) in mergeAfterExtract/restoreOrDrop, and log (not swallow) restore failures. Optionally fsync before rename for crash consistency.

User-supplied runId reaches filesystem read/delete paths without the assertSafeSegment validation the codebase mandates

- **File:** `packages/deepsec/src/sandbox/orchestrator.ts`
- **Lines:** 505, 539, 631
- **Slug:** path-traversal
- **Confidence:** high

checkStatus (line 505), collect (line 539) and deleteRunState (line 631) pass the CLI-supplied runId into state.ts's runPath(), which builds path.join(dataDir(projectId), 'sandbox-runs', `${runId}.json`) with NO validation. This violates the invariant documented in core/src/paths.ts ('Reject empty, '.', '..', absolute paths, null bytes, and any path separator... Used at every entry point that joins user-supplied segments') — core's own runMetaPath validates runId via assertSafeSegment, but the sandbox state module does not. Consequently `deepsec sandbox status --run-id ../../../foo` or `deepsec sandbox collect --run-id ../../../foo` will JSON.parse (loadRunState) or unlink (deleteRunState, line 631) arbitrary *.json files outside the project data dir as the invoking user. Exposure is limited to the local operator (who already owns the filesystem), so this is a defense-in-depth/inconsistency bug rather than a remotely exploitable vulnerability — but any future caller that passes a non-CLI-derived runId would inherit a real traversal. Related: loadRunState also throws an unhandled ENOENT for unknown runIds, crashing the CLI with a raw stack trace instead of a friendly error.

Recommendation: Call assertSafeSegment(runId, 'runId') inside state.ts runPath() (mirroring core runMetaPath), or validate at the CLI boundary before calling checkStatus/collect; handle missing state files with a clean error message.

---

extraArgs are re-split on whitespace, destroying values containing spaces and allowing silent override of injected flags

- **File:** `packages/deepsec/src/sandbox/orchestrator.ts`
- **Lines:** 48, 49
- **Slug:** other-arg-splitting
- **Confidence:** high

buildSandboxInvocation appends passthrough CLI args with `tail.push(...arg.split(/\s+/).filter(Boolean))` (lines 48-50). Any operator argument whose value legitimately contains whitespace — e.g. --filter "src/(my dir)" or a matcher pattern with spaces — is silently tokenized into multiple argv entries, changing what the worker subcommand receives (wrong filter, unexpected flag/value pairs). Because passthrough args are appended after the injected --project-id/--root/--manifest, a passthrough token can also silently override those host-injected flags on the worker's parser, misdirecting the run (e.g. pointing --root elsewhere) with no error. Input is operator-controlled, so this is a correctness bug rather than a security boundary issue, but it makes sandbox runs silently diverge from what the operator typed.

Recommendation: Pass extraArgs through as pre-split argv entries (array passthrough) instead of joining/re-splitting strings, or require an explicit separator convention (e.g. `--`-style) and reject tokens containing whitespace.

Workflow mutates process-wide cwd for the entire async setup run

- **File:** `packages/deepsec/src/setup/coordinator.ts`
- **Lines:** 353, 489
- **Slug:** other-global-cwd-mutation
- **Confidence:** medium

runSetupWorkflow() calls process.chdir(workspaceDir) (L353) and only restores it in the finally block (L489). The setup phases await long-running agent, scan, and network work, so any concurrently executing code in the same process (timers, plugin callbacks, signal handlers) resolves relative paths against the workspace during that window. If the workflow throws before the try block is entered or the finally itself is skipped due to process teardown, the process is left in the wrong directory. This is a process-global side effect from a library-style API and is a known source of subtle path bugs rather than a directly exploitable flaw.

Recommendation: Avoid process.chdir; pass absolute workspace paths (already resolved at L297) into every phase, or confine the chdir to synchronous boundaries with guaranteed restoration.

TOCTOU between repository file listing and fingerprinting crashes setup on repo churn

- **File:** `packages/deepsec/src/setup/fingerprint.ts`
- **Lines:** 52, 54
- **Slug:** other-race-condition
- **Confidence:** medium

repositoryFingerprint() iterates a file list produced by listRepositoryFiles() and calls fs.statSync(absolute) / fs.readFileSync(absolute) for each entry. The listing and the per-file stat/read are not atomic: if any listed file is deleted, renamed, or becomes unreadable between listRepositoryFiles() and the fingerprint loop (normal editor writes, git checkout, IDE/refactor activity, or another concurrently running deepsec command against the same repo), statSync throws ENOENT/EACCES and the exception propagates out of services.fingerprint() in setup/coordinator.ts (called outside any runPhase() wrapper), aborting the whole setup workflow with an unhandled error instead of a resumable phase failure. statSync/readSync also race each other (size captured from a pre-rename stat while reading post-rename content), producing a fingerprint that doesn't correspond to any consistent repo state, which can spuriously invalidate or preserve checkpoints. The impact is a crash-abort of setup on plausible concurrent repo modification, requiring a full rerun; no data corruption or security impact.

Recommendation: Make fingerprinting resilient to churn: wrap per-file statSync/readFileSync in try/catch and either skip vanished files (recomputing the file set) or retry the whole listing+fingerprint pass when the file set changes; alternatively stat with lstat semantics once and open with O_NOFOLLOW-style consistency checks. Surface the failure as a resumable phase error rather than an unhandled crash.

TOCTOU race in stale-lock takeover can break mutual exclusion (two concurrent setups)

- **File:** `packages/deepsec/src/setup/lock.ts`
- **Lines:** 38, 43, 44, 45
- **Slug:** other-race-condition
- **Confidence:** medium

When the lock file exists but its owner is dead, acquireSetupLock() (L39-44) reads the owner record and then unconditionally calls fs.unlinkSync(file) before retrying the openSync('wx'). The unlink is not bound to the record that was read (no fd/inode verification), so two processes that both observe the same stale lock can interleave: process A unlinks the stale lock and creates its own fresh lock; process B, having already decided the lock was stale, then unlinks A's newly created lock file and successfully creates its own. Both processes now return from acquireSetupLock() believing they hold the lock, while A's lock file has been deleted. Two

concurrent setup workflows then run against the same workspace: interleaved non-locked writes to data//setup/setup-state.json (last-writer-wins checkpoint corruption, phase digests that no longer match reality) and duplicated expensive AI processing (the 'process' phase costs ~$0.30/file per the project docs). The release callback partially limits the damage (it only unlinks if pid matches), but the mutual-exclusion guarantee itself is broken. The stale-lock path is realistic: it occurs whenever a previous setup was killed (SIGKILL/crash) and the operator or an automation loop retries.

Recommendation: Make the takeover atomic: open the lock file, read and verify the owner PID from the open fd, and delete via the fd (or use a lock-directory created with mkdir's atomicity plus a content check, or flock/fallocate-based locking). At minimum, re-read the file after unlink and confirm it is still the same stale record (compare pid/startedAt) before creating a replacement, and re-verify with a fresh openSync('wx') that also checks whether the unlinked path was recreated between unlink and create.

Lock staleness relies solely on PID liveness: PID reuse or corrupt lock records block setup permanently

- **File:** `packages/deepsec/src/setup/lock.ts`
- **Lines:** 23, 31, 39
- **Slug:** other-logic-bug
- **Confidence:** medium

acquireSetupLock() treats a lock as stale only when the recorded PID is not alive (L23-31, L39). There is no age/timeout and no validation of the record shape. Two failure modes: (1) The owning process died but its PID was recycled by an unrelated process — processIsAlive() returns true and every subsequent setup attempt fails with SETUP_ALREADY_RUNNING until the user manually deletes .deepsec-setup.lock. On long-lived machines with many short-lived processes (CI containers, agents spawning children), PID reuse within the lifetime of an orphaned lock is plausible. (2) If the lock record is corrupt or hand-edited (e.g. {"pid":0} or {"pid":"abc"}), processIsAlive() misbehaves: process.kill(0, 0) succeeds (signal 0 to the caller's process group is a valid permission check) and a non-numeric PID throws a non-ESRCH error which the catch block treats as 'alive' (error?.code !== 'ESRCH' → true). In both cases the lock is considered held forever with no automatic recovery path, producing an availability outage of the setup workflow.

Recommendation: Validate the parsed LockRecord (pid must be a positive integer, startedAt a parseable timestamp) and treat invalid records as stale. Add a staleness timeout (e.g. consider the lock stale if startedAt is older than a configurable TTL) in addition to PID liveness, and verify liveness against the recorded command line where possible to reduce PID-reuse false positives.

Terminal-line sanitizer leaves DEL, lone ESC, and C1 control characters in persisted output

- **File:** `packages/deepsec/src/setup/output.ts`
- **Lines:** 5, 6, 8
- **Slug:** other-control-char-injection
- **Confidence:** medium

normalizeTerminalLine() (L5-15) only strips full ANSI CSI/OSC sequences via ANSI_SEQUENCE, U+0008, and U+0000. It does not remove: U+007F (DEL), a lone trailing ESC that no longer matches the sequence regex, C1 controls such as U+009B/U+009D (8-bit CSI/OSC equivalents that some terminals honor), or other C0

bytes (U+000B, U+001C-U+001F). Package-manager and agent stderr is attacker-influenced content (scanned repo text echoed in tool output), and the sanitized lines are persisted into the setup JSONL log and replayed by the TUI and by coding agents reading the log. A crafted line can therefore smuggle cursor/color/OSC control sequences into downstream terminal consumers, contradicting the module's own comment that child output is 'persisted as text, never replayed as terminal control'.

Recommendation: After stripping ANSI sequences, remove all remaining C0 controls except tab (e.g. replace /[\u0000-\u0008\u000B-\u001F\u007F]/g) and strip C1 range U+0080-U+009F, then apply truncation.

String.replace special replacement patterns corrupt generated config when values contain '$'

- **File:** `packages/deepsec/src/setup/workspace-config.ts`
- **Lines:** 21, 50, 59
- **Slug:** other-logic-bug
- **Confidence:** high

reconcileWorkspaceConfig/upsertConfigLine build replacement strings by embedding JSON.stringify of user-supplied values (model name, agentType, thinkingLevel, ModelRoute fields) and pass them as the replacement argument of String.prototype.replace (e.g. `source.replace(new RegExp(...), line‘)‘atL21,‘source.replace(/ai − line − regex/m,‘{routeLine})` at L50-52, and the defaultAgent replace at L59). In JS, '&', '`', "$'", '$$' and '$< name >′ $in a string replacement are expanded as special patterns. A value like a model id 'foo$'-&-bar' or a route baseUrl containing "$'" would splice the matched line / the remainder of the config file into the rewritten deepsec.config.ts, producing a corrupted or semantically wrong config (setup then fails to loadConfig or silently uses wrong settings). Values come only from local CLI flags/checkpoint/existing config, so there is no remote attacker path — this is a robustness bug, not a security vulnerability.

Recommendation: Escape or bypass special replacement patterns: pass a function as the replacement (source.replace(regex, () => `${line}`)) or escape '$' as '$$' before interpolating values into replacement strings.

Abort listeners added to the shared quota-abort signal per batch are never removed

- **File:** `packages/processor/src/agents/claude-agent-sdk.ts`
- **Lines:** 303, 595
- **Slug:** other-resource-leak
- **Confidence:** high

investigate() (claude-agent-sdk.ts:303) and revalidate() (claude-agent-sdk.ts:595) each register `signal.addEventListener("abort", () => abortController.abort(), { once: true })` on the processor's shared quotaAbort.signal (processor/src/index.ts:596, 627, 1163 — one controller reused for every batch in a run), but neither generator ever removes the listener; the generator simply returns after emitting results. runClaudeSetupTask (L236-239) shows the intended pattern (removeEventListener in finally), but investigate/revalidate omit it. Since { once: true } only fires if abort actually happens, one listener (plus its captured AbortController) leaks per batch — hundreds over a large run — growing the listener set past Node's default max of 10 and triggering MaxListenersExceededWarning spam on every later

emitter operation, plus memory retention. Contrast: the pi plugin (pi-sdk.ts runPiPrompt/runToollessFollowUp) correctly removes its listeners in finally.

Recommendation: Capture the handler in a variable and call signal.removeEventListener("abort", handler) in a finally block (or use AbortSignal.any / pass the AbortController down), mirroring runClaudeSetupTask.

AgentSession (and its event subscription) leaked when a batch fails with a quota-classified error

- **File:** `packages/processor/src/agents/pi-sdk.ts`
- **Lines:** 949, 1124
- **Slug:** other-resource-leak
- **Confidence:** high

In both investigate() and revalidate(), the pi session created by createPiSession is disposed on every failure path EXCEPT the quota short-circuit. In investigate (pi-sdk.ts:947-950) and revalidate (pi-sdk.ts:1122-1125): `if (quotaSource) { throw new QuotaExhaustedError(quotaSource, lastError); }` throws while `session` still holds a live AgentSession — the session was created in the try block above and runPiPrompt already exited via its catch — and no `session?.dispose()` runs before the throw. Every sibling path disposes (the `!resultText` throw at 956/1131, each JSON-repair failure at 911/986/1003, and the normal exit at 1039/1208), so this is an oversight, not a design choice. Quota exhaustion is precisely the condition under which the processor aborts ALL in-flight batches (QuotaExhaustedError → shared quotaAbort), so one exhausted credential leaks one undisposed session per concurrently-failing batch: each holds the in-memory session state, the subscribe() closure, and runtime resources until GC. Compare the claude/codex plugins, which have no per-session teardown to skip.

Recommendation: Dispose the session before throwing: `session?.dispose(); throw new QuotaExhaustedError(quotaSource, lastError);` in both investigate() and revalidate(), or wrap the attempt loop in try/finally that disposes any live session on abnormal exit.

Synchronous git spawn inside buildRevalidatePrompt blocks the shared event loop for up to 10s per file while other agent batches stream concurrently

- **File:** `packages/processor/src/agents/shared.ts`
- **Lines:** 1033
- **Slug:** other-blocking-io
- **Confidence:** medium

buildRevalidatePrompt (shared.ts:1033-1041) calls spawnSync('git', ['log', ...], { timeout: 10_000 }) once per file in each revalidate batch (batchSize default 5). The processor runs batches concurrently (params.concurrency, processor/src/index.ts:592, 1099) in a single Node process; a spawnSync freezes the entire event loop for up to 10 seconds per file — up to ~50s per revalidate invocation and longer across a run — stalling every in-flight agent HTTP stream (SSE parsing, keepalive, SDK timeouts) and all progress emission for all concurrent batches. With several revalidate invocations queued this can trip transport timeouts in sibling batches, converting a metadata lookup into spurious batch failures and retries (each retry costing real model spend). The git call itself is safely argv-formed with a '--' pathspec separator and validated cwd, so this is a reliability bug, not an injection vector.

Recommendation: Replace spawnSync with an async execFile/promisified spawn (awaited while building the prompt), or gather git history for the whole batch in a single async pass outside the per-file loop.

---

enrich() rewrites FileRecords from a stale snapshot with no locking — concurrent runs lose findings

- **File:** `packages/processor/src/enrich.ts`
- **Lines:** 242, 209
- **Slug:** other-lost-update-race
- **Confidence:** medium

enrich() loads all FileRecords once (loadAllFileRecords), then enrichOne() blindly calls writeFileRecord(record) (enrich.ts:242) per record, taking no per-project/per-record lock and never re-reading. If `deepsec enrich` runs concurrently with `deepsec process`/`revalidate` (or another enrich), the enrich write clobbers the other run's freshly written findings/verdicts/analysisHistory with its stale copy (and vice versa: the other run's write erases the enrichment). The codebase explicitly defends the process-vs-process case (acquireProcessLock + isReclaimableLock) and the sandbox-vs-sandbox case (mergeFileRecord), but enrich participates in neither serialization.

Recommendation: Re-read each record immediately before writing and merge enrichment into the fresh copy (enrich only sets gitInfo, which merge logic already preserves), or serialize enrichment writes under the same locking used by process().

Committer index keys use git's quoted path form — enrichment silently missing for non-ASCII/special-character paths

- **File:** `packages/processor/src/enrich.ts`
- **Lines:** 140, 223
- **Slug:** other-logic-bug
- **Confidence:** medium

buildGitCommitterIndex keys the map by the raw output of `git log --name-only` (enrich.ts:140), but git C-quotes paths containing non-ASCII or special characters by default (core.quotePath=true): a file `src/café.ts` is emitted as `"src/caf\303\251.ts"`. The lookup `committerIndex.get(record.filePath)` (enrich.ts:223) uses the unquoted glob path, so every quoted path misses and the file silently gets an empty committers array — enrichment data (and any ownership decisions based on it) is quietly absent for a meaningful fraction of real-world repos. git config could also differ per environment, making keys additionally unstable.

Recommendation: Run git with `-c core.quotePath=false` (and normalize separators), or unquote C-style quoted paths when building/lookup up the index.

Revalidate persists verdicts with blind writeFileRecord and no locking — concurrent runs silently lose verdicts/findings

- **File:** `packages/processor/src/index.ts`
- **Lines:** 1360, 1346

- **Slug:** other-lost-update-race
- **Confidence:** medium

runRevalidateInvocation mutates in-memory FileRecords loaded once at start and writes them back with writeFileRecord (index.ts:1360) with no per-record lock (the code only serializes the RunMeta phase: 'revalidate doesn't lock FileRecords'). Two concurrent revalidate invocations, or revalidate racing a process() run (which claims and later rewrites records outside revalidate's knowledge), or racing the streaming sandbox extract+merge (withExtractLock protects sandbox-vs-sandbox merges, but no local command participates in it), produce classic lost updates: whichever stale copy is written last erases the other run's verdicts, analysisHistory entries, or newly-found findings.

Recommendation: Re-read the record immediately before each write and merge (like mergeFileRecord does), or take the same per-project/per-record claim mechanism process() uses for revalidate's writeback phase.

Force modes (reinvestigate / direct filePaths) bypass active locks and clobber in-flight runs

- **File:** `packages/processor/src/index.ts`
- **Lines:** 529, 542, 549, 556
- **Slug:** other-lock-bypass-clobber
- **Confidence:** medium

The claim loop's guard `if (!isOurs && !isFreelyClaimable && !inForceMode) continue;` (index.ts:549) is skipped entirely when `inForceMode` is set (index.ts:529: `!!reinvestigate || params.filePaths !== undefined`). A `--reinvestigate <N>` wave run or `process --diff` run will steal files that another live run currently owns (status 'processing' with a live PID) — precisely the race the isReclaimableLock comment calls 'catastrophic'. Both agents then investigate the same file concurrently (double model spend), and the last writer overwrites the loser's findings and analysisHistory because both write their full in-memory snapshot. The wave-marker docs promise idempotency, which does not hold while a previous run is still active.

Recommendation: In force modes, still refuse to claim records whose lock owner is alive (reuse isReclaimableLock for the ownership check while bypassing only the status/pending filter), or re-read and merge the fresh record before the post-investigation write.

Revalidation verdicts persisted without enum validation — invalid values silently erase the whole finding on next load

- **File:** `packages/processor/src/index.ts`
- **Lines:** 1234, 1242
- **Slug:** other-unvalidated-model-output
- **Confidence:** high

parseRevalidateVerdicts returns `parsed as RevalidateVerdict[]` with no schema validation, and reconcileVerdicts only matches identifiers — it never validates `verdict` or `adjustedSeverity` values. The apply loop then writes them raw: `finding.revalidation = { verdict: m.verdict.verdict, ... }` (index.ts:1234) and `finding.severity = m.verdict.adjustedSeverity` (index.ts:1242). An out-of-enum verdict string (model flakiness, or steering via prompt-injected finding descriptions — verdict

reasoning/descriptions originate from untrusted repo content) is persisted; the strict read-side revalidationSchema/findingSchema (nested z.enum) then fails, and salvageFileRecord drops the ENTIRE finding with only a console warning — silently removing a real finding from all reports/metrics on the next load. Unknown verdict strings are also miscounted as 'uncertain' (else-branch totalUncertain++). Contrast with the investigate path, which field-validates via findingSchema.safeParse and runs an in-session repair loop.

Recommendation: Validate each verdict against a zod schema (verdict enum, adjustedSeverity enum, duplicateOf requirement) in parseRevalidateVerdicts or reconcileVerdicts; drop/repair invalid verdicts before applying them to findings, and never set finding.severity to an unvalidated value.

Verdict-to-finding matching by exact title misassigns and miscounts duplicate titles

- **File:** `packages/processor/src/triage.ts`
- **Lines:** 204, 215
- **Slug:** other-logic-bug
- **Confidence:** high

`batch.find((b) => b.finding.title === verdict.title)` (triage.ts:204) matches the FIRST finding with that title. When two findings in the same batch share a title (common after dedupe-tolerant investigation, e.g. two matchers surfacing the same issue with identical wording), every verdict for that title is applied to the first item only; the second finding is never triaged yet is counted: totalTriaged++ and the p0/p1/p2/skip counters increment per verdict (triage.ts:215-219), not per distinct finding triaged. The summary line and completeRun stats therefore overstate triage coverage and hide the skip.

Recommendation: Include filePath and findingId in the triage output contract and match on those (fall back to first-unmatched-by-title), and count per uniquely-triaged finding.

Cap-detection window bleeds across statements, suppressing candidates for genuinely uncapped LLM calls

- **File:** `packages/scanner/src/matchers/agent-loop-no-cap.ts`
- **Lines:** 50, 51, 52, 53
- **Slug:** other-logic-bug
- **Confidence:** medium

The matcher intends to check 'the call's argument block' for a cap, but the implementation scans the current line plus the next 30 lines of ANY content (lines 50-53). Any occurrence of maxSteps/maxTurns/stopWhen/signal/abortSignal/timeout within that window suppresses the candidate — including a cap belonging to a subsequent, different LLM call, a nested fetch()'s unrelated `timeout:`, or even a comment mentioning `maxSteps`. Result: a genuinely uncapped `streamText`/`generateText`/`query()` call (the DoS/cost-burn condition this matcher exists to catch) goes unreported whenever any cap-shaped token appears anywhere in the following 30 lines. Conversely, an uncapped call whose options object merely extends past the 30-line window is reported despite having a cap. This produces both false negatives and false positives in the scanner's highest-cost finding class. (Note: the scanner-flagged hits on this file itself — 'weak cipher algorithm' at L18 and 'agent loop' at L21-27/L41 —

are self-scan false positives: L18 is the `description:` line matched by the unanchored /DES|RC4|Blowfish/i regex, and L21-27/L41 are the examples array and a comment, not live LLM calls.)

Recommendation: Bound the cap search to the call's own argument object: track brace/paren balance from the call's opening `(` and stop at the matching close, rather than a fixed 30-line window. Within that block, match `maxSteps`/`maxTurns`/`stopWhen`/`abortSignal` keys exactly and drop `timeout:` and bare `signal:` from the cap set (they don't bound agent iterations).

Scope-detection heuristic has a too-narrow context window and matches bare words, producing both missed and spurious cross-tenant cache findings

- **File:** `packages/scanner/src/matchers/cache-key-scope.ts`
- **Lines:** 45, 53, 54, 55
- **Slug:** other-logic-bug
- **Confidence:** medium

The matcher decides a cache key is properly scoped by searching a 3-line context (1 line before, 2 after, line 53) for the words userId/teamId/auth./session./projectId etc. (lines 54-57). This fails in both directions: (1) False negatives — the dominant real-world pattern `const userId = session.user.id` (or `const teamId = params.teamId`) is defined well above the cache call, so genuinely unscoped keys like `redis.get(`settings:${name}`)` are treated as scoped and never flagged, defeating the cross-tenant cache-poisoning detection this matcher exists for. (2) False positives — an unrelated nearby line that merely mentions `auth.` or `session.` (a comment, an import path, a different variable) suppresses the finding. Additionally, the `KV get with colon-separated key` pattern (line 45) `\.get\s*\(\s*[^]*:/` matches ANY `.get(` with a template key containing a colon — Map.get, headers.get, searchParams.get — flooding candidates with non-cache lookups. (The scanner's own hits on this file are all self-scan false positives: 'weak cipher algorithm' at L11/L52 comes from the unanchored /DES|RC4|Blowfish/i regex matching 'de**s**cription' and 'inclu**des**', and the non-atomic-read-delete hits at L14/16/19/24/25 flag the examples array snippets, not real Redis code.)

Recommendation: Widen the scope-variable search to the enclosing function body (or at least ~15 lines before the cache call), and require the scope variable to be interpolated into the key itself (e.g. test whether an identifier matching /userId|teamId|tenantId/ appears inside the template literal) rather than anywhere in nearby lines. Restrict the colon-key pattern to known cache client receivers (redis/cache/store) instead of any `.get(`.

Cron-route detection uses a raw /cron/i substring test on the path and misses common handler forms

- **File:** `packages/scanner/src/matchers/cron-secret-check.ts`
- **Lines:** 30, 33
- **Slug:** other-logic-bug
- **Confidence:** high

Line 30 decides a file is a cron route via `/cron/i.test(filePath)` — a case-insensitive substring test with no word boundaries. Any path containing 'cron' as a fragment of another word is treated as a cron route: e.g. `src/synchronize.ts`, `lib/synchronized-lock.ts`, `cronify.ts`. Any exported GET/POST

function in such a file is then emitted as 'Cron route handler without CRON_SECRET validation', generating false positives in a matcher declared as noiseTier 'precise'. In the opposite direction, the `hasHandler` regex (line 33) only recognizes `export (async )?function GET|POST` and `export (const )?(GET|POST)` — it misses `export default function handler()`, `export const PUT/DELETE/PATCH` (used by some cron providers), and Next.js route files that re-export handlers (`export { GET } from ...`), so real unauthenticated cron endpoints can go entirely unflagged. (The scanner's hits on this file — 'weak cipher algorithm' at L7 and 'missing-auth' Next.js handler at L17/24/32 — are self-scan false positives: L7 is the `description:` line matched by /DES|RC4|Blowfish/i, and L17/24/32 are the examples array strings, not live route handlers.)

Recommendation: Match cron paths with a segment-aware pattern (e.g. /(^|[/.-])cron(s)?([/.-]|$)/i) and detect the env var via a broader allowlist (CRON_SECRET, CRON_KEY, etc.). Extend hasHandler to `export default`, PUT/DELETE/PATCH, and re-export forms, and emit one candidate per unflagged handler instead of breaking after the first.

Source classifier's safe/static branch is unreachable — every hit is mislabeled because /html/i always matches 'dangerouslySetInnerHTML'/'innerHTML'

- **File:** `packages/scanner/src/matchers/dangerous-html.ts`
- **Lines:** 36, 37, 38, 41, 43, 45, 47, 48
- **Slug:** other-dead-classifier-branch
- **Confidence:** high

In dangerousHtmlMatcher, the source classifier computes isFromProps = /props.|children|content|html|body|text|message/i.test(context). Every matched line necessarily contains 'dangerouslySetInnerHTML' or '.innerHTML', both of which contain the substring 'html', so isFromProps is always true. Because the label chain is ordered isFromSearchParams -> isFromFetch -> isFromProps -> isSafeScript || isStaticString -> fallback, the dedicated static/safe-content branch (label 'dangerouslySetInnerHTML with static/safe content (weak candidate)') is dead code and can never fire. Genuinely static content such as `<div dangerouslySetInnerHTML={{ __html: "<h1>Hi</h1>" }} />` is mislabeled 'from props (trace source)'. The problem is compounded by the higher-priority heuristics: isFromFetch matches any 'await' and isFromSearchParams matches bare 'query'/'params'/'req.' anywhere in the ±3/+5-line context window, so static or safe content is frequently escalated to 'fetched data (MEDIUM RISK)' or 'URL/request params (HIGH RISK)' based on unrelated nearby words. Impact is degraded scanner-output quality (systematically misleading labels and inflated candidate noise flowing into the AI triage stage), not an exploitable vulnerability in deepsec itself — the intent expressed by the isSafeScript/isStaticString variables and their label branch is provably never realized.

Recommendation: Remove the overly broad 'html' alternative from isFromProps (or anchor it, e.g. /\bprops.|children|content\b|body\b|message\b/), and/or reorder the classification chain so isSafeScript/isStaticString are evaluated before the speculative source heuristics. Also tighten isFromFetch (require 'await fetch' or 'response.' rather than any 'await') and isFromSearchParams (require 'searchParams' or 'req.query' rather than bare 'params'/'query').

Greedy-wildcard regex runs over full uncapped file content, enabling CPU-exhaustion DoS from a crafted scanned repo

- **File:** `packages/scanner/src/matchers/debug-endpoint.ts`
- **Lines:** 46, 52, 13
- **Slug:** other-regex-dos
- **Confidence:** medium

All automated flags on this file are false positives: the 'HTTP handlers', 'x-debug headers', 'process.env' dumps, and 'weak cipher' hits are inside the `examples` array (string fixtures consumed only by **tests**/matcher-examples.test.ts, which passes them as content to matcher.match() and never evaluates them) or are the matcher's own detection regex literals (e.g. /x-debug|x-test-|x-internal/ at line 47). The file contains no live endpoint, crypto, or env access. The one real defect is performance: `hasDebugCode = /debug.*endpoint|test.*endpoint|dev.*only|development.*only/.test(content)` (line 46) is computed on the ENTIRE file content before the early-return gate at line 52, for every file matching the broad `**/api/**/*.{ts,tsx,js}` / `**/app/api/**/*.{ts,tsx}` globs (lines 9-14). Because `.` does not match newlines but the engine (RegexScannerDriver in packages/scanner/src/index.ts) reads files whole with readFileSync and no size cap, an adversarial repo (which the threat model explicitly treats as untrusted input) can ship a single multi-megabyte .ts line containing many 'debug'/'test'/'dev' tokens and no terminator word: each literal occurrence forces `.*` to backtrack across the whole line, yielding O(k·n) ~ O(n²) work that stalls the scan run. Root cause is shared with the engine's lack of a file-size cap; this matcher is one of the most exposed because its heaviest regexes execute pre-gate on full content.

Recommendation: Cap file size before matching (skip or truncate large files in RegexScannerDriver), and/or bound the hasDebugCode/hasDebugHeaders regexes to per-line or fixed-window evaluation instead of full content. The same hardening applies engine-wide to other matchers with greedy wildcards.

Lookahead window crosses case/break boundaries, misattributing handlers and producing systematic false positives

- **File:** `packages/scanner/src/matchers/event-handler-mismatch.ts`
- **Lines:** 21, 84, 87, 88, 93
- **Slug:** other-logic-bug
- **Confidence:** high

The matcher scans the 5 lines following each `case` label (line 84: `lines.slice(i + 1, i + 6)`) without stopping at `break;` statements or the next `case` label. In the extremely common switch shape where each case body is short, the window captures the NEXT case's handler call. Example: `case "user.added": addUser(u); break; case "user.removed": removeUser(u); break;` — for `case "user.added"` (opposites include "remove"), the window contains `removeUser(`, so correct code is flagged as a copy-paste bug and the finding's lineNumbers (line 93: `[i + 1]`) attribute the next case's function to the wrong event. Symmetrically, real mismatches where the handler call sits more than 5 lines below the case label (multi-line bodies) are missed (false negatives). Aggravating factors: the catch-all filePattern `**/*.{ts,js}` (line 21) runs this against every JS/TS file in scanned repos, and the matcher self-flags its own `examples` array (the template literals at lines 24-45 were themselves reported as candidates). Impact is limited to candidate noise since downstream AI triage re-checks verdicts, but the systematic misattribution can mislead the triage agent and burn model budget.

Recommendation: Truncate the lookahead window at the first `break;`, `return`, or `case`/`default` keyword encountered; skip lines that start a new `case`; and consider excluding the matcher's own examples

via the existing test/stub file filter (e.g. add a guard for files under the matchers directory or strip the examples array before matching).

Weak-cipher pattern /DES|RC4|Blowfish/i lacks word boundaries — matches 'description', 'design', etc., flooding the candidate pipeline

- **File:** `packages/scanner/src/matchers/insecure-crypto.ts`
- **Lines:** 31
- **Slug:** other-regex-precision-bug
- **Confidence:** high

The insecure-crypto matcher's weak-cipher detection regex at line 31 is /DES|RC4|Blowfish/i with no word-boundary anchors (\b). Because 'DES' is a case-insensitive substring of extremely common words, the pattern fires on virtually any JS/TS file containing 'description', 'design', 'destroys', 'indexing' variants like 'DEScribe', etc. The scanner's own dogfood scan demonstrates the bug: it flagged this file's own line 7 (`description: "Weak cryptographic algorithms..."`) as a 'weak cipher algorithm' hit, solely because the word 'description' contains 'DES'. Consequence: near-universal false-positive candidate generation across scanned repos. Each candidate file is batched to an AI agent for triage (an expensive LLM call), so this bug inflates processing cost and drowns real findings in noise. The other patterns in this matcher (createHash('md5'), createCipher(), bounded hmac/digest comparisons) are precise; only the cipher-alternation regex is broken. Note /createCipher\s*(/ correctly does NOT match createCipheriv(because it requires an immediate opening paren, so the deprecated-API pattern is fine as-is.

Recommendation: Anchor the alternation with word boundaries and require cipher-shaped context, e.g. /["']\b(DES|RC4|Blowfish)(-ECB|-CBC)?\b["']/ or /\b(DES|RC4|Blowfish)\b\s*(?=["']\),]\)/ — at minimum add \b on both sides of each alternative. Also consider anchoring to assignment/call context (e.g. createCipheriv\(\s*["']) to match how the pattern is actually used in real code.

Default-export route handler regex misses named default exports (false negatives in entry-point discovery)

- **File:** `packages/scanner/src/matchers/js-nextjs-route-handlers.ts`
- **Lines:** 59, 60
- **Slug:** other-detection-gap
- **Confidence:** high

The scanner's own 'weak cipher' and 'missing-auth' flags on this file are false positives (they are example strings and regex literals inside a matcher definition, per the project's known false-positive list). However, the matcher has a real logic bug. The default-export detection regex at line 59, /export\s+default\s+(async\s+)?(function\s*)?(/, requires an opening parenthesis immediately after the optional 'function' keyword. It therefore matches only anonymous default exports ('export default function (req, res)' or 'export default (req) => ...') and silently fails on named default exports such as 'export default function handler(req, res) { ... }' or 'export default async function handler(...)'. Named default exports are the canonical and overwhelmingly common form for Next.js Pages Router API routes (pages/api/*.ts) and Lambda-style handlers. Because the matcher's stated purpose is comprehensive HTTP entry-point discovery ('Every route.ts file is an HTTP endpoint', 'Also flag default exports (Lambda-style handlers)'), these files produce

zero candidates, flow through the pipeline unflagged, and their real vulnerabilities are never triaged by the AI processor — a systematic detection blind spot rather than an exploitable flaw in deepsec itself. Note the examples array only exercises the anonymous forms, so tests would not catch this.

Recommendation: Allow an optional identifier between 'function' and the parameter list, e.g. /export\s+default\s+(async\s+)?(function\s+[A-Za-z_$][\w$]\s)?(/, and add inline examples/test cases for 'export default function handler(req, res)' and 'export default async function handler(...)'. Also consider matching reference-style defaults ('export default handler;') via a follow-up heuristic.

Line-scoped matching systematically misses multi-line SQL calls and several interpolation forms — false negatives for the highest-severity SQLi class

- **File:** `packages/scanner/src/matchers/py-sql-raw.ts`
- **Lines:** 37, 38, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 99
- **Slug:** other-logic-bug
- **Confidence:** high

All patterns are applied per-line via the shared regexMatcher helper (matchers/utils.ts), but Python code routinely formats DB calls across lines: `cursor.execute(\n f"SELECT * FROM users WHERE id = {user_id}"\n)` places the f-string on a different line from `execute(` and matches nothing, so the exact SQL-injection pattern this matcher exists to catch goes undetected in commonly formatted code. Coverage is also uneven across drivers: `.format()` and `+` concatenation detection exists only for `cursor.execute` (lines 63-72), not for `session.execute`, asyncpg `conn.execute/fetch*`, Django `objects.raw`, or `connection.cursor().execute` — so `session.execute("... {} ...".format(val))` and `User.objects.raw("..." + name)` are silently missed. The bounded quantifiers `[^"']{0,400}` additionally cause the %-format/.format/+ patterns to stop matching when the SQL literal exceeds 400 characters (long, realistic queries). Finally, the file-path skip `/\b(?:tests?|migrations)\b/i` fails on underscore-separated names (`test_utils.py` does not match `\btests?\b`), so test helpers are scanned while true `tests/` directories are skipped. (The scanner's hits on this file are self-scan false positives: 'weak cipher algorithm' at L20 is the `description:` line matched by the unanchored /DES|RC4|Blowfish/i regex in insecure-crypto.ts, and the non-atomic-read-delete hits at L25-32/L99 flag the examples array strings and the `db.execute` regex source line, not live code.)

Recommendation: Either switch this matcher to a windowed multi-line scan (join each call site with its following lines until the closing paren, as agent-loop-no-cap does) or pre-normalize content to collapse parenthesized continuations before line matching. Add .format()/+ concatenation patterns for session.execute, conn.execute/fetch*, objects.raw, and connection.cursor().execute, and raise/restructure the 400-char cap (e.g. anchor the interpolation operator to the end of the statement).

Per-line greedy-wildcard URL regexes over uncapped content from every JS/TS file enable CPU-exhaustion DoS from a crafted scanned repo

- **File:** `packages/scanner/src/matchers/url-regex-validation.ts`
- **Lines:** 31, 9, 34
- **Slug:** other-regex-dos
- **Confidence:** medium

All automated flags on this file are false positives: the 'ssrf' / 'new URL from request data' / 'bypassable regex' hits are inside the `examples` array (string fixtures used only by the matcher self-test, never evaluated) or are the matcher's own detection regex literals. No live request handling exists here. The real defect is performance: the hasUrlRegex condition's first operand, `/https?.*\.+|https?.*.*/.test(line)` (circa line 31), executes on EVERY line of EVERY file matching `**/*.{ts,tsx,js,jsx}` (line 9) — the broadest glob in the matcher family — before any cheaper gate. On a crafted single multi-megabyte line containing many 'http' tokens but no '.+' or '.*' terminator, each literal occurrence forces `.*` backtracking across the line (O(k·n)); additionally, each `new URL(` occurrence (hasUncheckedUrl branch) builds and scans a 5-line joined window, repeating O(n) work per occurrence. The scan engine (RegexScannerDriver in packages/scanner/src/index.ts) reads files with no size cap, and the threat model designates the scanned repo as untrusted input, so an adversarial repo can stall or severely slow a scan run. Non-security (tool availability only), hence BUG severity.

Recommendation: Cap file/line size before matching in RegexScannerDriver (skip or truncate oversized files), reorder the hasUrlRegex condition so cheap anchors (e.g. /.test|.match|RegExp/ or a single 'http' probe) gate the greedy-wildcard regex, and bound the hasUncheckedUrl window join. Note the same uncapped-content pattern exists in sibling matchers.
